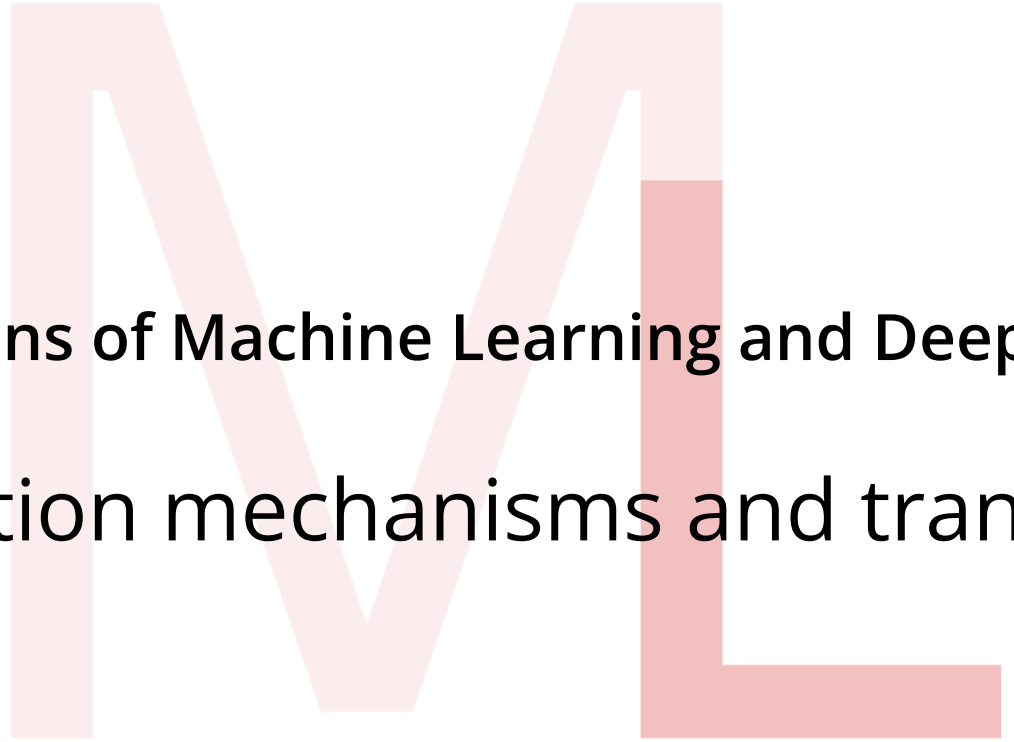
Foundations of Machine Learning and Deep Learning

# 11. Attention mechanisms and transformers



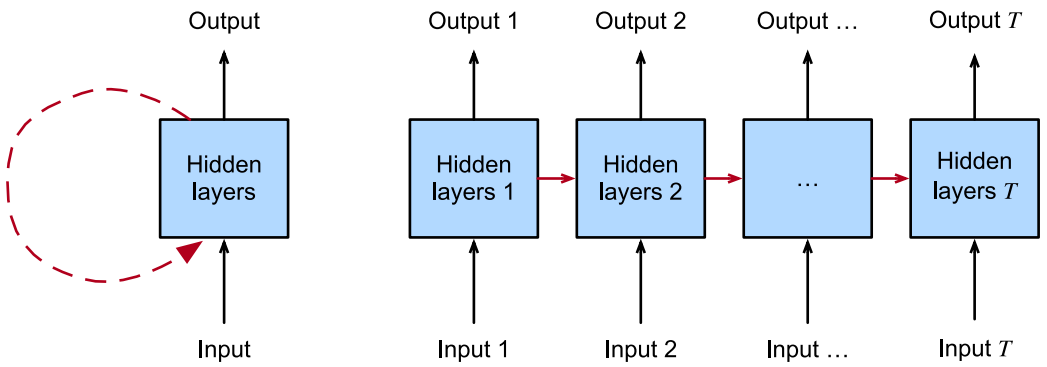

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Introduction

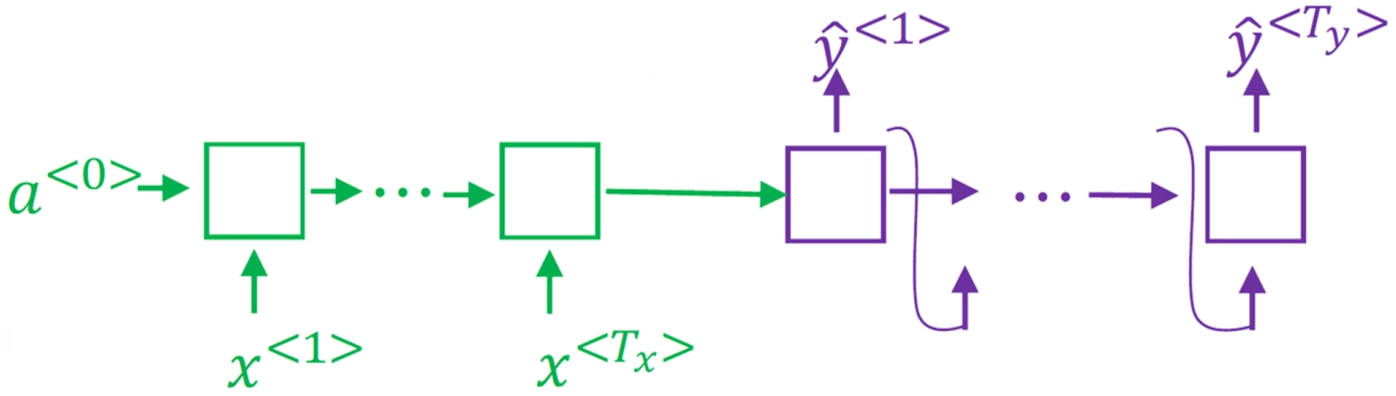
- So far…
  - We have processed sequences with RNNs (one step at a time)
  - The hidden state carries information forward
  - Useful for tasks like classification




  - For tasks like machine translation (e.g. English to Spanish):
    - Seq2Seq RNN (many-to-many)

# Introduction

- **Encoder-decoder**
  - Encoder reads English input sentence
  - Decoder generates Spanish output sentence

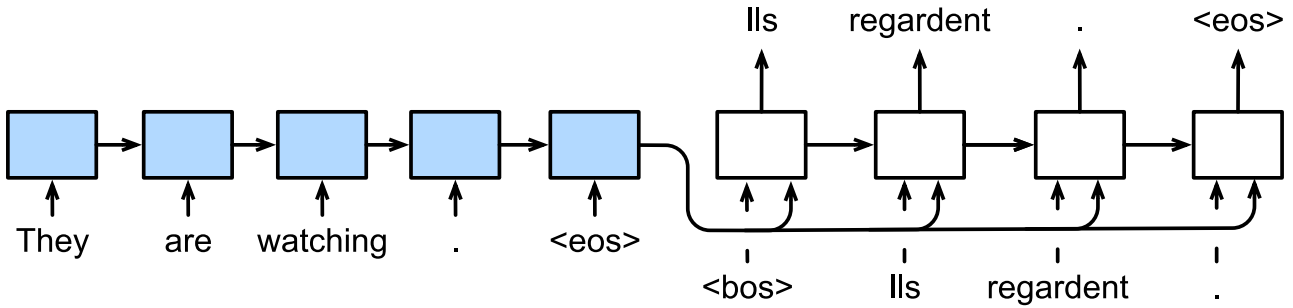



  - We want to compute $argmax_y\ P\left(y_1, \ldots, y_{T_y} \middle| \mathbf{x}\right)$

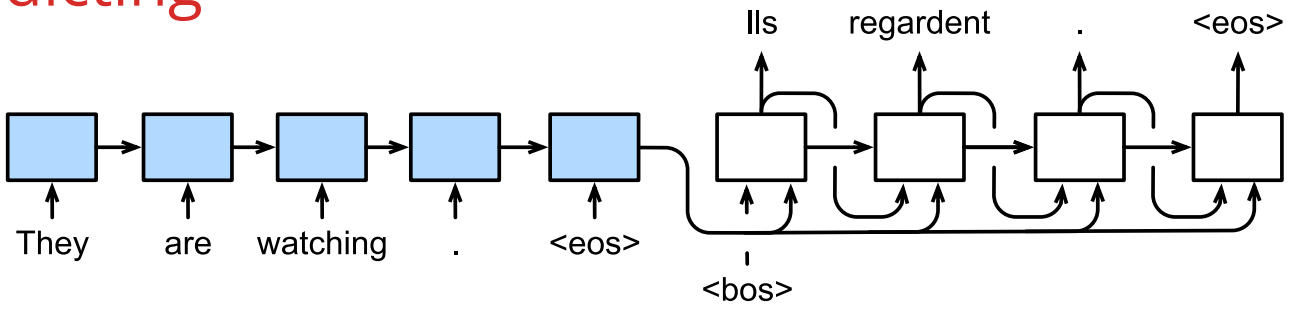
Vector $h_{T_x}$ summarizes entire input

# Introduction

- **Encoder-decoder**
  - Training
    - Decoder is conditioned upon preceding “ground-truth” tokens




  - Predicting




Later in the master’s we will see:

- Strategies to find the best output sequence
  - Greedy decoding (pick most likely word each step)
  - Beam search (keep multiple candidates)
- Evaluation metrics
  - BLEU score

# Introduction

- **Encoder-decoder: Problem**
  - Context vector is a bottleneck
  - Works for short sentences: “The black cat” → “El gato negro”
  - Fails for long, complex sentences (information loss)
    - Long-range dependencies (early words forgotten)
    - Word reordering across languages (e.g., adjectives)

Solution: **Attention**

English (input)

“*Bilbo, who had never seen a dragon or even a real battle before, was beginning to feel very small and rather frightened*”

Spanish (output)

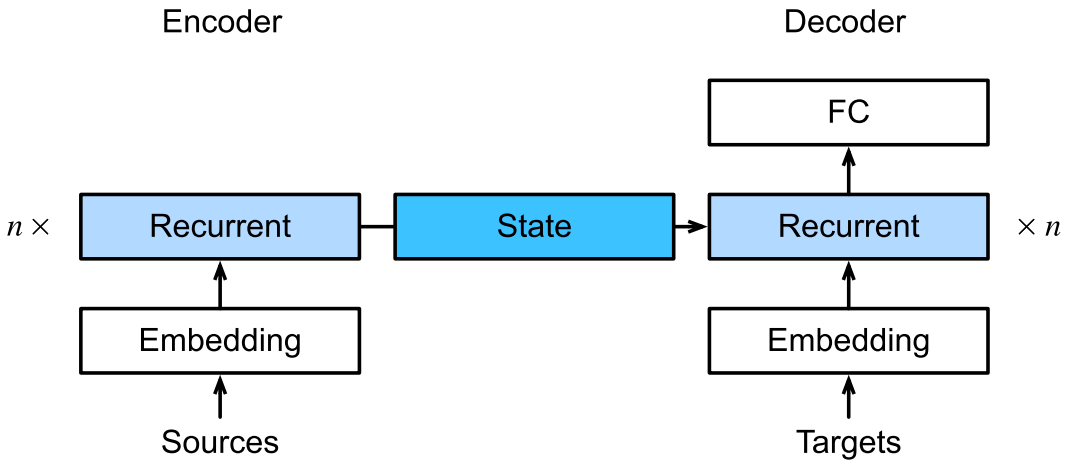
“*Bilbo, que nunca había visto un dragón, ni siquiera una verdadera batalla, empezaba a sentirse muy pequeño y bastante asustado*”

# Outline

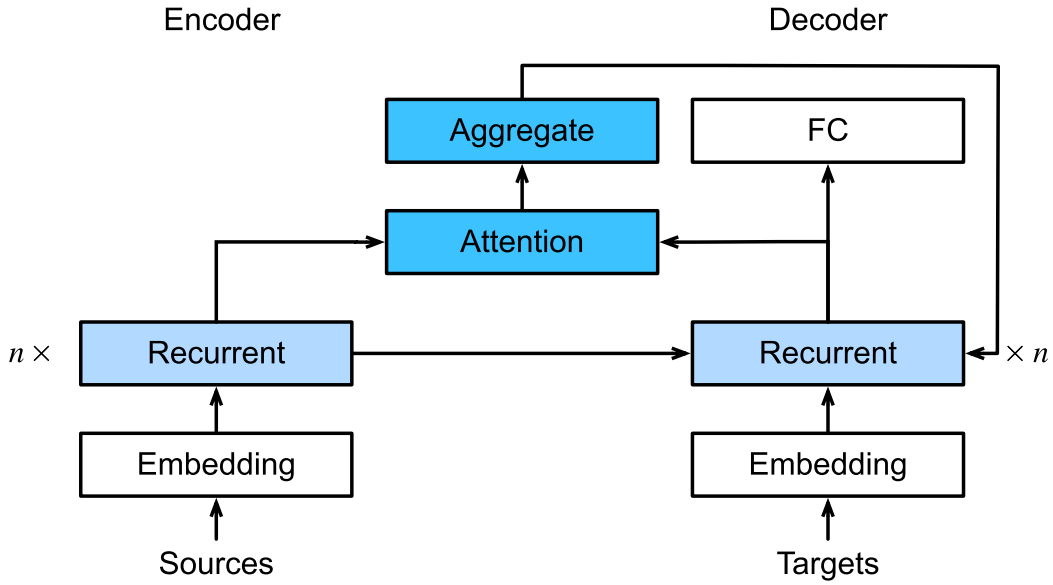
- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Attention in RNNs

- **Attention (Bahdanau et al. 2014)** - https://arxiv.org/pdf/1409.0473
  - Does not rely on one fixed context vector
    - The decoder can access all enconder states




No attention




Attention

# Attention in RNNs

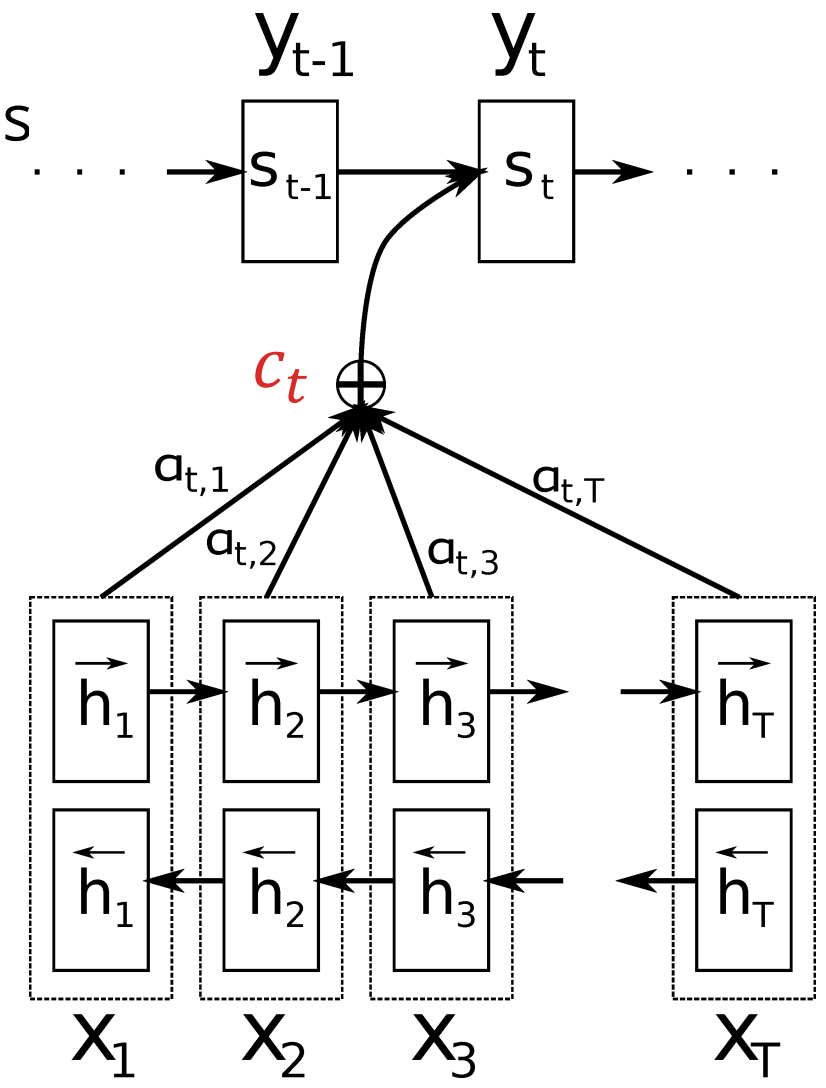
- **Attention (Bahdanau et al. 2014)** - https://arxiv.org/pdf/1409.0473
  - At each decoding step:
    - Compare current decoder state with all encoder states
    - Assign weights (focus on the most relevant words)
    - Build a context vector tailored to this step
      - When generating "*asustado*" focuses on "*Bilbo*" and "*frightened*"
    - Context vector is a weighted sum of each hidden state:

$$c_t = \sum_i \alpha_{t,i} h_i$$

      - The weights ($\alpha$) can be seen as the "attention" that the decoder pays to each word in the encoder to predict the next word

# Attention in RNNs

- **Attention (Bahdanau et al. 2014)**
  - The encoder produces a sequence of hidden states $\{h_1, h_2, \ldots, h_{T_x}\}$ for the input tokens
  - At each decoding time step $i$
    - The decoder has a previous hidden state $s_{i-1}$
  - We compute an attention score (alignment)
    - $e_{ij} = a(s_{i-1}, h_j)$ $a$ is a feedforward NN that returns a scalar
  - We convert the scores to weights via softmax:
    - $$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$
  - We compute the context vector $$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$
  - ...



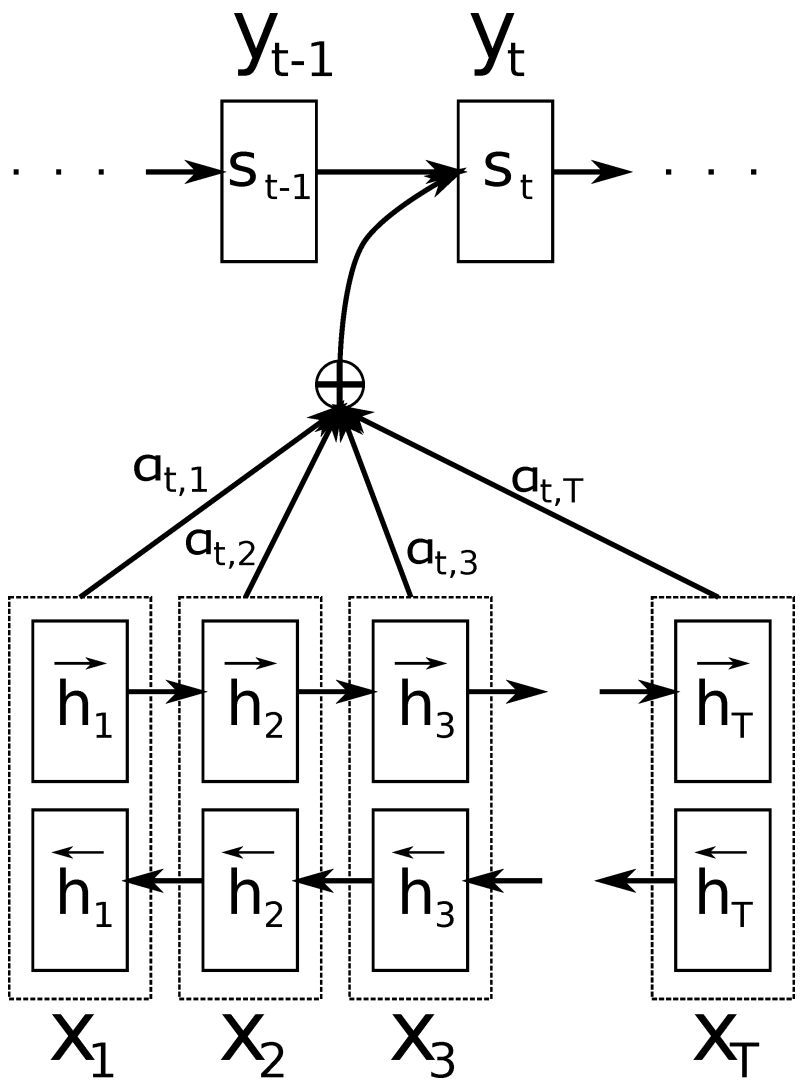
# Attention in RNNs

- **Attention (Bahdanau et al. 2014)**
  - …
  - Compute decoder hidden step at step $i$
    - $s_i = f(s_{i-1}, y_{i-1}, c_i)$

$f$ is the RNN cell update
(in the original paper a GRU)

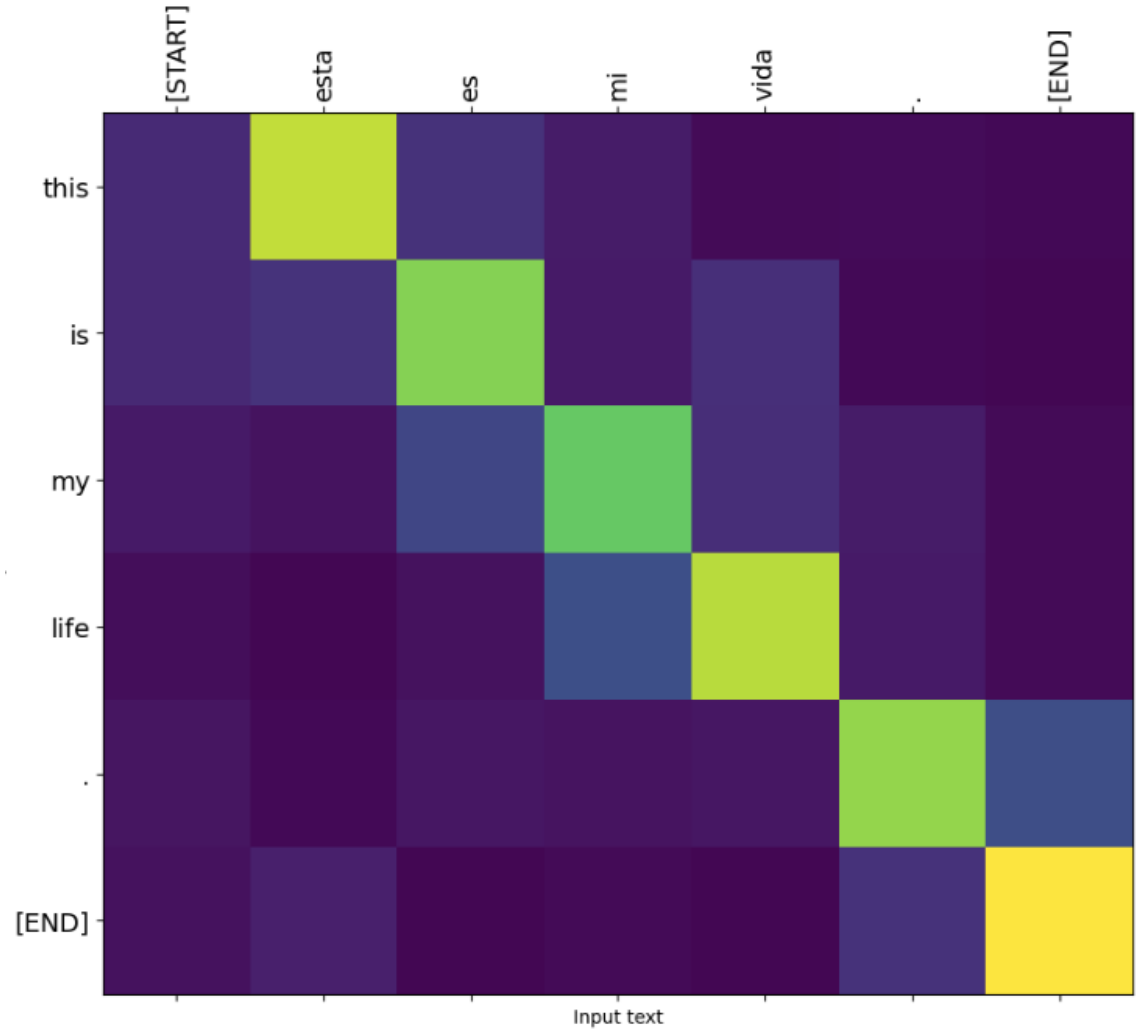
  - Compute the probability for output $y_i$
    - $P(y_i \mid y_1, \ldots, y_{i-1}, \boldsymbol{x}) = g(y_{i-1}, s_i, c_i)$

$g$ is the function that combines decoder state, previous target word, and context to predict the next word (in the original paper feed-forward + softmax)



# Attention in RNNs

- **Attention (Bahdanau et al. 2014) – Intuition**
  - Each $\alpha_{ij}$ is the attention weight
    - It tells how much the decoder should "look at" source word $x_j$ when generating the next target word $y_i$



# Attention in RNNs

- **Attention (Bahdanau et al. 2014) – Limitations**
  - Only encoder-decoder links
    - Attention connects decoder states to encoder states
    - Within the encoder/decoder, information still flows sequentially through RNNs, so token-token relations are indirect
  - RNN dependency
    - Attention is tied to recurrent networks: sequential computation
    - Training and inference is slow
  - Context decoding
    - Context vectors must be recomputed at every decoding step

Solution: **self-attention**

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
  - Self-attention
  - Multi-head self-attention
- Transformer architecture: Encoder
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Attention in Transformers – Self-attention

- **Self-attention**
  - Proposed in "*Attention is All You Need*" (Vaswani et al., 2017)
    - https://arxiv.org/pdf/1706.03762
  - Core ideas
    - Each word can directly attend to all other words in the sequence, regardless of distance
    - Attention is not decoder-encoder, but also encoder-encoder
    - For each word, I can now choose where to look (what words in the sequence), there is no longer a hidden state (we remove recurrence)
  - It enables highly parallelizable trainig and better modeling for long-range dependencies

# Attention in Transformers – Self-attention

- Each token embedding generates its own **query, key** and **value**
- Intuition: Think of a dictionary or a search engine
  - Query (Q): what I'm looking for
    - Attention: token asking "what other tokens are relevant to me?"
  - Keys (K): labels of items in the database
    - Attention: each token's "identifier" used for matching
  - Values (V): the actual contents I retrieve
    - Attention: the information carried by each token
- Process:
  - Compare query with all keys (similarity score)
  - Attention = weighted average of all values, based on query-key similarity

# Attention in Transformers – Self-attention

- **Queries, Keys and Values**
  - Each token (e.g. word embedding) is a vector $\boldsymbol{x} \in \mathbb{R}^d$
  - We project $\boldsymbol{x}$ into three different spaces:
    - $q = W_Q \cdot \boldsymbol{x},\ q \in \mathbb{R}^{d_k}$ (query)
    - $k = W_K \cdot \boldsymbol{x},\ k \in \mathbb{R}^{d_k}$ (key)
    - $v = W_V \cdot \boldsymbol{x},\ v \in \mathbb{R}^{d_v}$ (value)

represent

what this token wants to know about others

how this token can be recognized / matched

the information this token carries to share

Example

- Token = “frightened”
- Its query *q* asks: “who explains me?”
- The keys *k* of other tokens act like labels.
  - “Bilbo” might have a key that strongly matches the query from “frightened”
- The values *v* are retrieved and the representation of “frightened” is a combination of values from all tokens (weighted by how much its query match their keys)

**Dot-product + softmax**

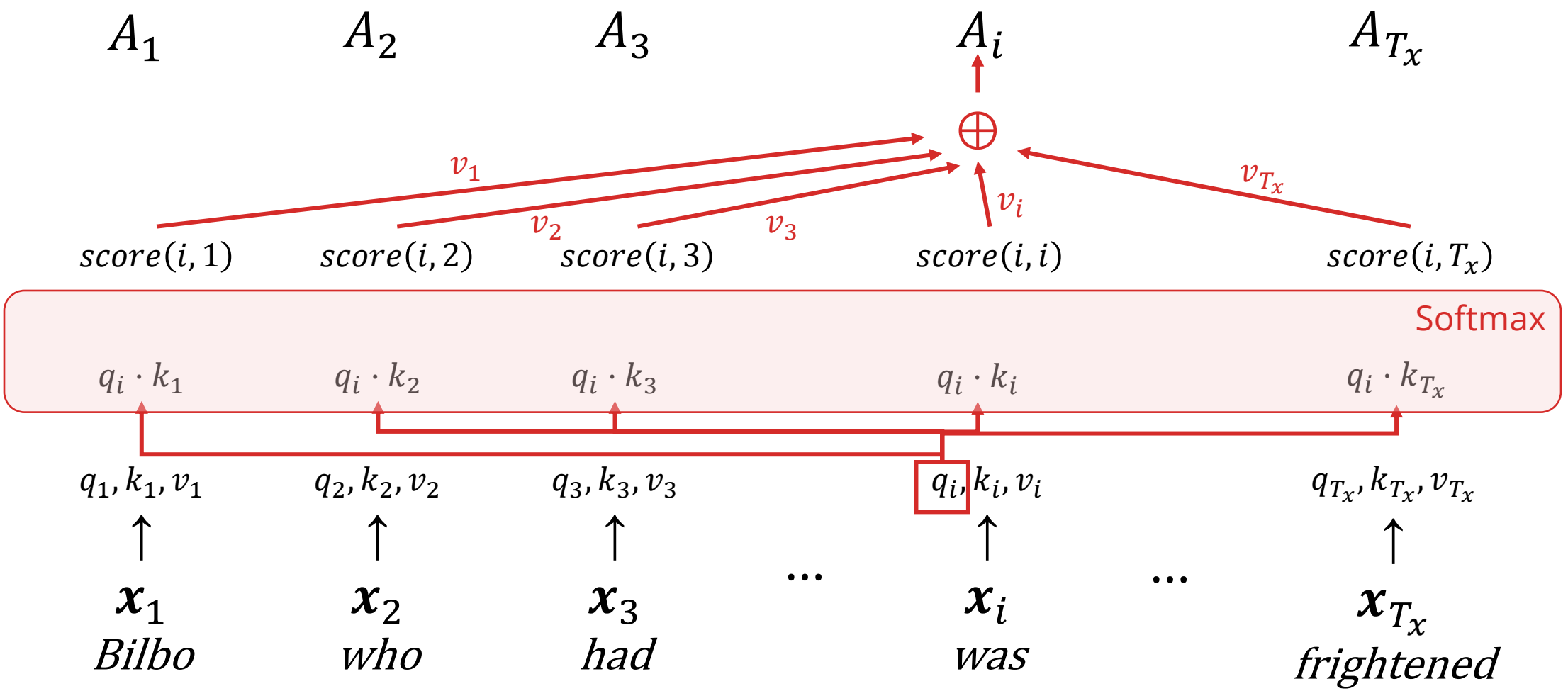
# Attention in Transformers – Self-attention

- **Attention based representation of token $i$: $A(q, K, V)$**
  - Matching score between query $i$ and key $j$ is the scalar product and then softmax
    - $\text{score}(q_i, k_j) = \frac{\exp(q_i \cdot k_j)}{\sum_p \exp(q_i \cdot k_p)}$
    - It tells us how much attention to pay to token $j$
  - Use these matching scores to compute the weighted average of the values
    - $A_i = A(q, K, V) = \sum_j \frac{\exp(q_i \cdot k_j)}{\sum_p \exp(q_i \cdot k_p)} v_j$

$A_i$ is the vector that represents token $i$ in this context (attention-based representation)

# Attention in Transformers – Self-attention

- **Attention based representation of token $i$: $A(q, K, V)$**



# Attention in Transformers – Self-attention

- **Matrix form**
  - With the embedding matrix:
    - $X \in \mathbb{R}^{T_x \times d_{\text{model}}}$
  - We learn three projections:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

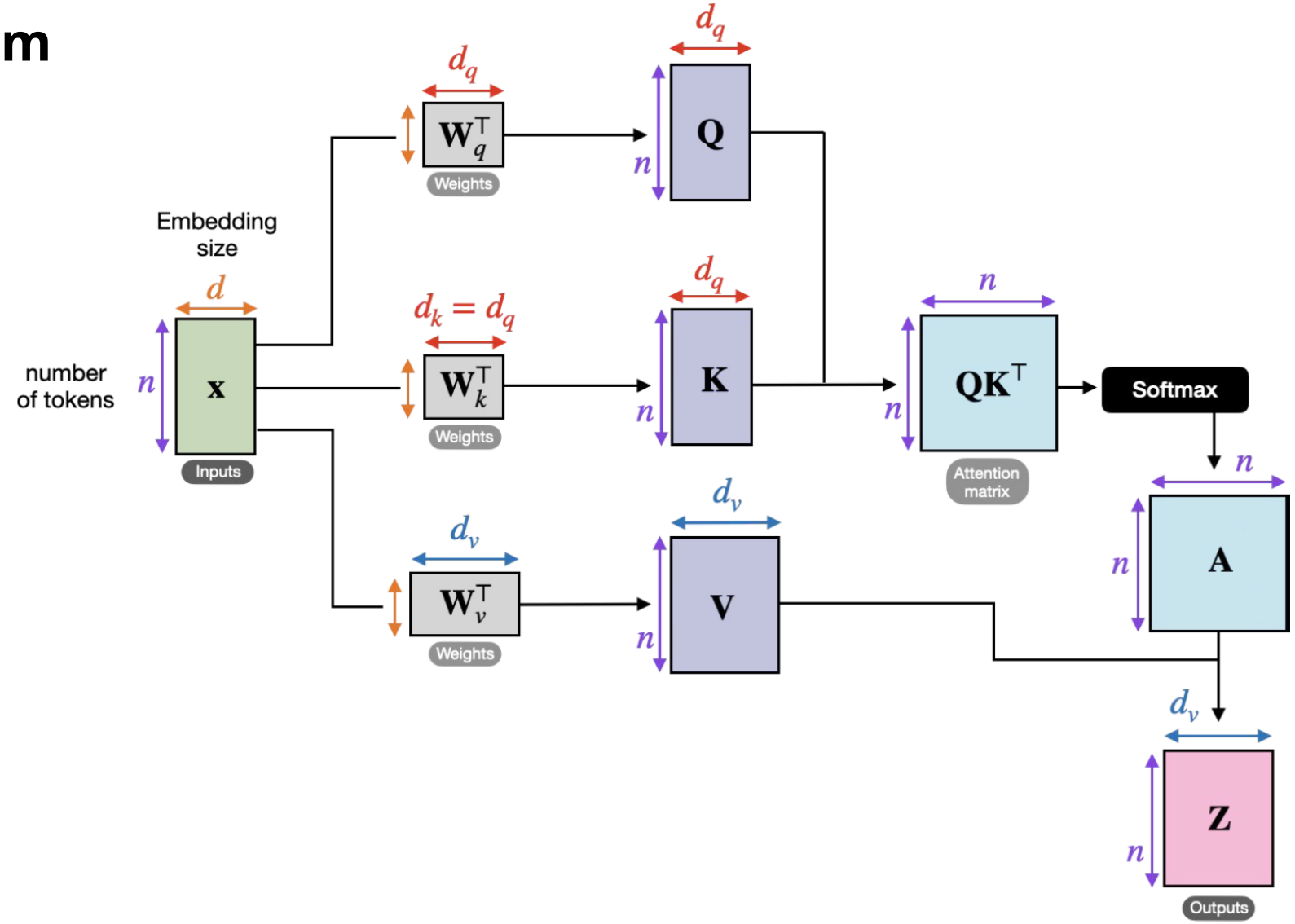
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

    - $Q, K \in \mathbb{R}^{T_x \times d_k}$
    - $V \in \mathbb{R}^{T_x \times d_v}$
  - Similarity: $S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T_x \times T_x}$
    - Row $i$ contains scores of token $i$ attending to all tokens.

Scaling by $\sqrt{d_k}$ avoids large magnitudes that would saturate softmax

# Attention in Transformers – Self-attention

- **Matrix form**



# Attention in Transformers – Self-attention

- **Learnable parameters**
  - Three projection matrices

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

    - These matrices are trained end-to-end with the rest of the model
    - No “hand-crafted” attention
      - The model learns how to query, match, and share information

# Attention in Transformers – Self-attention



- **Example: "The hungry cat chased the mouse"**
  - Setup
    - $d_{model} = 6$ (one-hot embedding per token)
    - $d_k = d_v = 2$
    - $X$ = the 6x6 identity matrix

$$W_Q = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0.5 & 0 \\ 0 & 1 \end{bmatrix} \quad W_K = \begin{bmatrix} 0.2 & 0 \\ 0.6 & 0 \\ 1.2 & 0 \\ 0 & 1 \\ 0.2 & 0 \\ 0.8 & 0 \end{bmatrix} \quad W_V = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**
  - Compute $Q = XW_Q, \quad K = XW_K, \quad V = XW_V$

$$Q = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0.5 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 0.2 & 0 \\ 0.6 & 0 \\ 1.2 & 0 \\ 0 & 1 \\ 0.2 & 0 \\ 0.8 & 0 \end{bmatrix} \quad V = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**
  - Scores

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad \sqrt{d_k} = \sqrt{2} \approx 1.414$$

$$S \approx \begin{bmatrix} 0.071 & 0.212 & 0.424 & 0 & 0.071 & 0.283 \\ 0.141 & 0.424 & 0.849 & 0 & 0.141 & 0.566 \\ 0 & 0 & 0 & 0.707 & 0 & 0 \\ 0.141 & 0.424 & 0.849 & 0 & 0.141 & 0.566 \\ 0.071 & 0.212 & 0.424 & 0 & 0.071 & 0.283 \\ 0 & 0 & 0 & 0.707 & 0 & 0 \end{bmatrix}$$

# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**
  - Softmax (row-wise) $P = \mathrm{softmax}(S)$

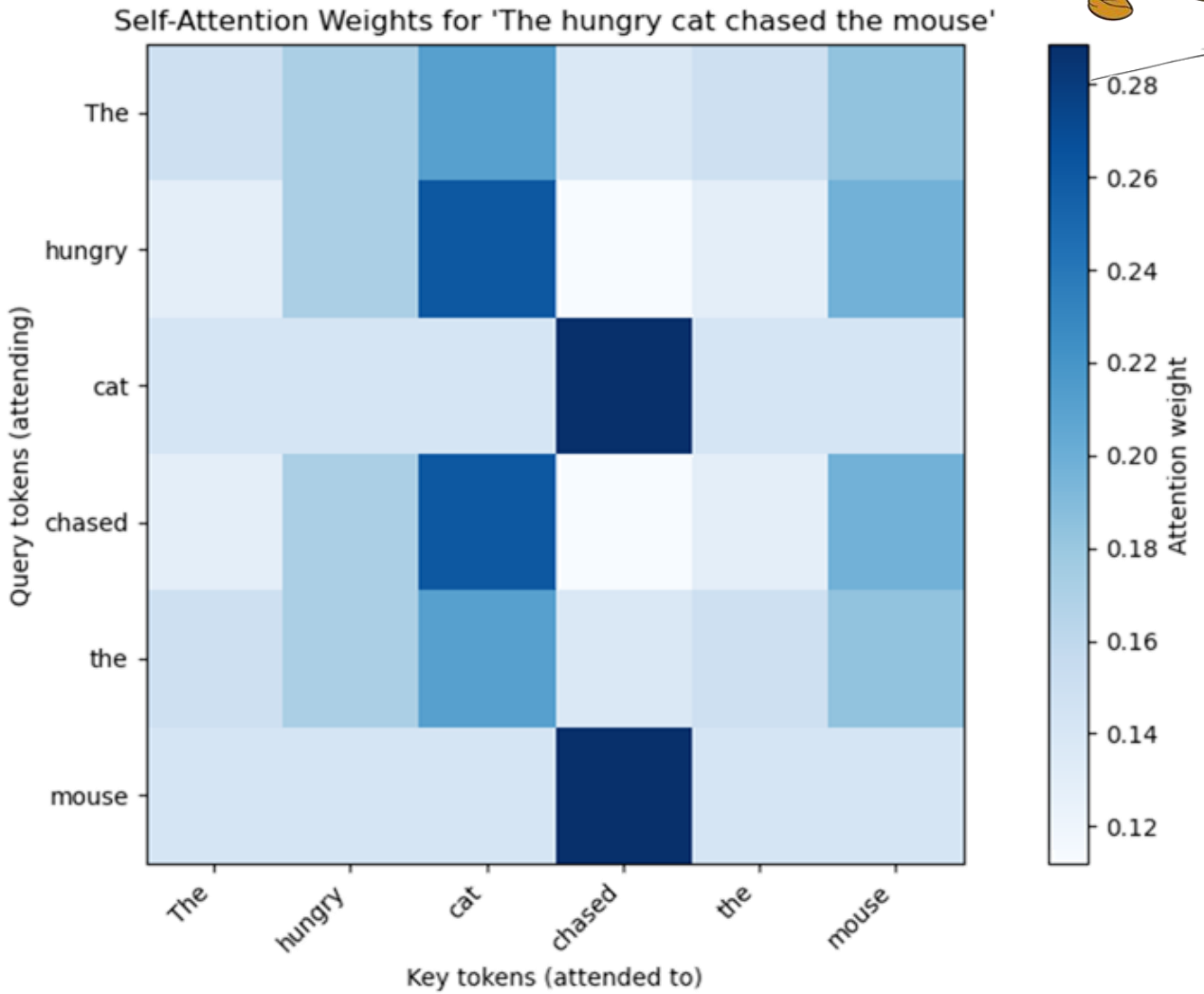
$$P \approx \begin{bmatrix} 0.148 & 0.171 & 0.211 & 0.138 & 0.148 & 0.183 \\ 0.129 & 0.171 & 0.262 & 0.112 & 0.129 & 0.197 \\ 0.142 & 0.142 & 0.142 & 0.289 & 0.142 & 0.142 \\ 0.129 & 0.171 & 0.262 & 0.112 & 0.129 & 0.197 \\ 0.148 & 0.171 & 0.211 & 0.138 & 0.148 & 0.183 \\ 0.142 & 0.142 & 0.142 & 0.289 & 0.142 & 0.142 \end{bmatrix}$$

# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**

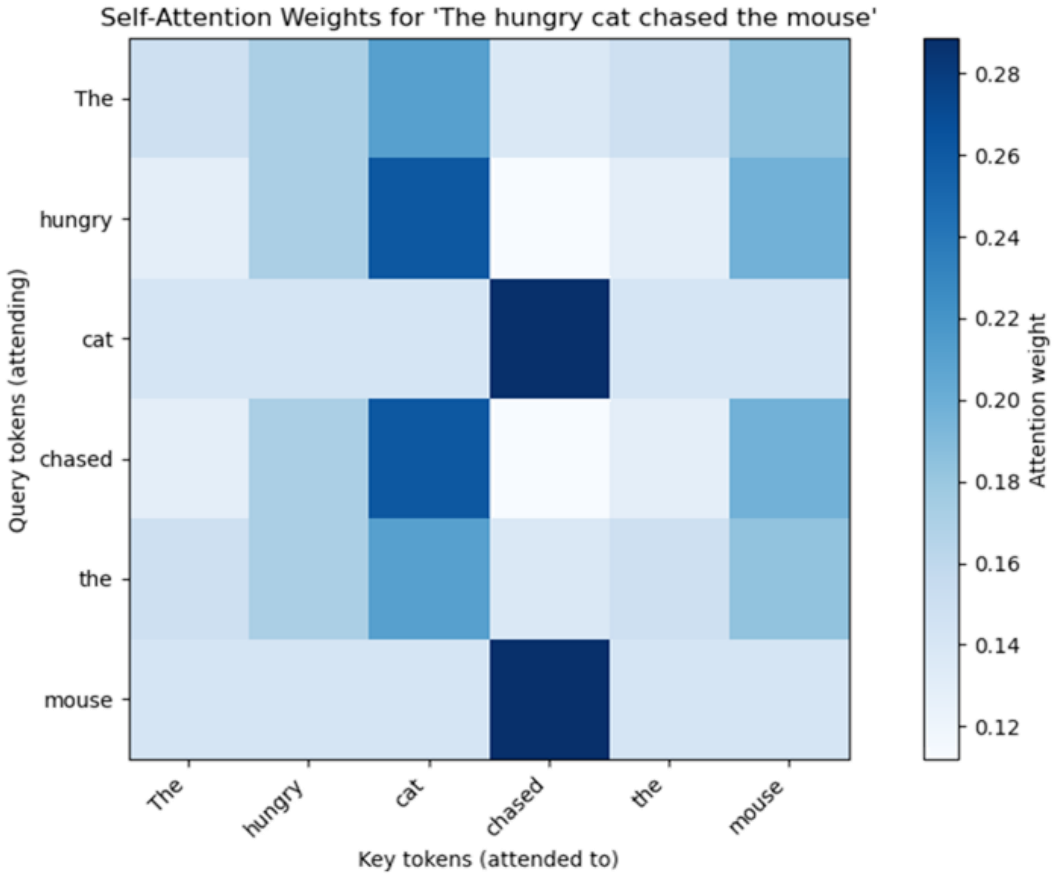
$$P \approx$$



# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**




- Hungry (adj) focuses more on “cat” than “mouse”
- Chased splits focus between “cat” (subject) and “mouse” (object)
- Cat and mouse focus most on the verb chased

# Attention in Transformers – Self-attention



- **Example: “The hungry cat chased the mouse”**
  - **Output (final representation of tokens)**

$$O = PV$$

$$O \approx \begin{bmatrix} 0.5653 & 0.5326 \\ 0.6300 & 0.5708 \\ 0.4269 & 0.5731 \\ 0.6300 & 0.5708 \\ 0.5653 & 0.5326 \\ 0.4269 & 0.5731 \end{bmatrix}$$

# Attention in Transformers – Self-attention

- **Summary**
  - Each token creates query, key and value vectors from its embedding
  - Attention lets a token compare itself to all other tokens and decide how much information to take from them
  - The output for each token is a weighted average of all values (tokens can integrate context across entire sequence)
  - Self-scores ensure tokens keep part of their own identity
  - This mechanism is permutation-invariant (it does not know word order)
    - We will add positional information
  - Advantage over RNNs:
    - Parallelizable (all tokens attend simultaneously)
    - Global context in one step (no sequential bottleneck)

# Attention in Transformers – Self-attention

- **Practical exercise:**
  - Implementing a self-attention class from scratch

```
class SelfAttention(nn.Module):
    def __init__(self, d_model, d_k, d_v):
```

- Create Linear layers self.W_Q, self.W_K, self.W_V (set bias to False)
- Create self.scale

```
    def forward(self, X):
```

- Compute Q, K and V
- Compute scales dot-product scores
- Compute softmax
- Weighted sum of values
- Return outputs

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
  - Self-attention
    - Multi-head self-attention
- Transformer architecture: Encoder
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Attention in Transformers – Multi-head self-attention

- **Multi-head attention**
  - A single attention layer projects into one query/key/value space
  - It may capture one type of relationship (e.g., subject ↔ verb)
  - But language has **multiple simultaneous relationships**:
    - Subject – verb: *Bilbo* ↔ *was beginning*
    - Verb constructions: *had* ↔ *seen*
    - Modifier – noun: *real* ↔ *battle*
    - Long – range: *Bilbo* ↔ *frightened*
  - We need "many heads" of attention
    - One head cannot capture all of these simultaneously

# Attention in Transformers – Multi-head self-attention

- **The idea of multi-head attention**
  - Multiple sets of Q, K, V matrices (each head has its own)
    - Each focusing on different patterns

$$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$$

  - Each head computes self-attention independently
  - Outputs are concatenated (along the feature axis)): $\text{Concat}(head_1, \ldots, head_h)$

  - There is a **new learnable matrix**: $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$
    - It mixes all the heads into one vector space (keeping dims consistent)

$$\boxed{\text{Concat}(head_1, \ldots, head_h)W_O}$$

# Attention in Transformers – Multi-head self-attention

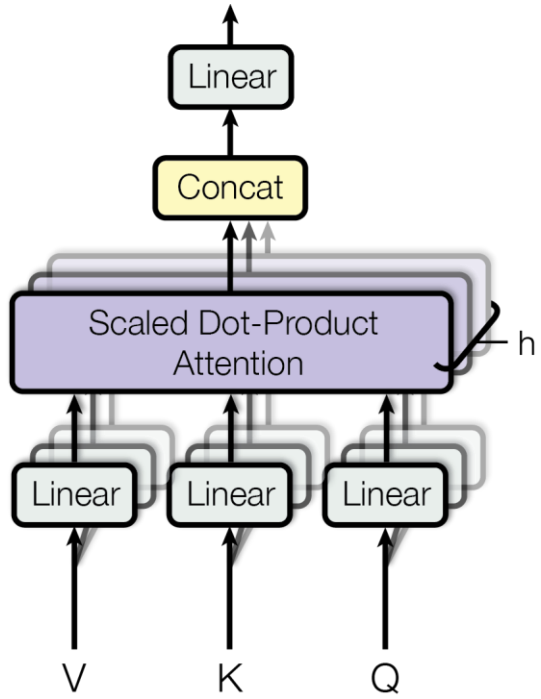
- **The equations of multi-head attention**
  - The main equation (per head) is:

$$\text{head}_i = \text{Attention}(Q^{(i)}, K^{(i)}, V^{(i)})$$

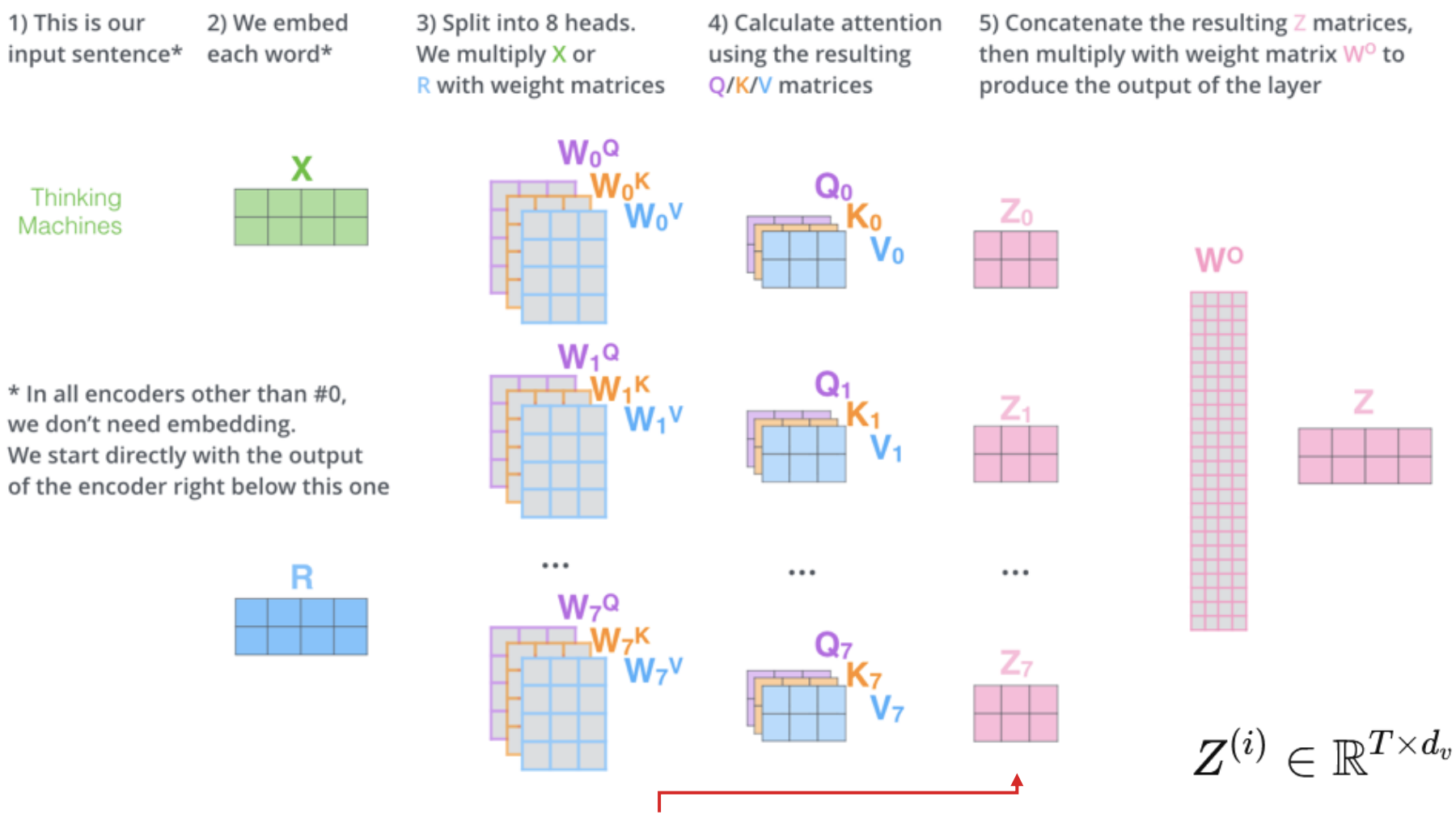
where $Q^{(i)} = XW_Q^{(i)}, \quad K^{(i)} = XW_K^{(i)}, \quad V^{(i)} = XW_V^{(i)}$

  - Then, the multi-head output is

$$\boxed{\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O}$$



# Attention in Transformers – Multi-head self-attention




$$Z^{(i)} \in \mathbb{R}^{T \times d_v}$$

$$Z = \text{Concat}(Z^{(1)}, \ldots, Z^{(h)}) \in \mathbb{R}^{T \times (h \cdot d_v)}$$

# Attention in Transformers – Multi-head self-attention

- **Learnable parameters**
  - Three projection matrices **per head**

$$W_Q^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_K^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

  - An additional projection matrix

$$W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

    - We want that the output has the same dimension as the input for a posterior step in the transformer achitecture

# Attention in Transformers – Multi-head self-attention



- **Example: "The hungry cat chased the mouse"**

  - Setup
    - $d_{model} = 4$
    - $d_k = d_v = 2$
    - $X$ = 6x4 matrix

| | |
|---|---|
| The | [1,0,0,0] |
| hungry | [0,0,1,0] |
| cat | [3,0,0,0] |
| chased | [0,2,0,0] |
| the | [1,0,0,0] |
| mouse | [2,0,0,0] |

  - Multi-head
    - $h = 2$ (two heads)
      - $Z^{(1)} = Attention(Q^{(1)}, K^{(1)}, V^{(1)})$
      - $Z^{(2)} = Attention(Q^{(2)}, K^{(2)}, V^{(2)})$

*computing…*

$$Z^{(1)} = \begin{bmatrix} 0.85 & 0.57 \\ 1.00 & 0.33 \\ 0.65 & 0.93 \\ 1.72 & 0.04 \\ 0.86 & 0.58 \\ 0.67 & 0.91 \end{bmatrix} \qquad Z^{(2)} = \begin{bmatrix} 1.16 & 1.33 \\ 1.50 & 1.58 \\ 1.17 & 1.33 \\ 1.23 & 1.65 \\ 1.39 & 1.42 \\ 1.15 & 1.29 \end{bmatrix}$$

# Attention in Transformers – Multi-head self-attention



- **Example: "The hungry cat chased the mouse"**

$$Z^{(1)} = \begin{bmatrix} 0.85 & 0.57 \\ 1.00 & 0.33 \\ 0.65 & 0.93 \\ 1.72 & 0.04 \\ 0.86 & 0.58 \\ 0.67 & 0.91 \end{bmatrix} \qquad Z^{(2)} = \begin{bmatrix} 1.16 & 1.33 \\ 1.50 & 1.58 \\ 1.17 & 1.33 \\ 1.23 & 1.65 \\ 1.39 & 1.42 \\ 1.15 & 1.29 \end{bmatrix} \Rightarrow Z = \begin{bmatrix} 0.85 & 0.57 & 1.16 & 1.33 \\ 1.00 & 0.33 & 1.50 & 1.58 \\ 0.65 & 0.93 & 1.17 & 1.33 \\ 1.72 & 0.04 & 1.23 & 1.65 \\ 0.86 & 0.58 & 1.39 & 1.42 \\ 0.67 & 0.91 & 1.15 & 1.29 \end{bmatrix}$$

The
hungry
cat
chased
the
mouse

$h \times d_v$

- Now, with a learnt matrix $W_O$,

$$W_O = \begin{bmatrix} \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & -\frac{1}{2} & 0 \\ 0 & \frac{1}{2} & 0 & -\frac{1}{2} \end{bmatrix} \Rightarrow \text{Out} = Z\,W_O$$

$$\text{Out} \approx \begin{bmatrix} 1.005 & 0.950 & -0.155 & -0.380 \\ 1.250 & 0.955 & -0.250 & -0.625 \\ 0.910 & 1.130 & -0.260 & -0.200 \\ 1.475 & 0.845 & 0.245 & -0.805 \\ 1.125 & 1.000 & -0.265 & -0.420 \\ 0.910 & 1.100 & -0.240 & -0.190 \end{bmatrix}$$

$d_{model}$

# Attention in Transformers – Multi-head self-attention

- **Some notes on dimensions**
  - If we denote
    - B = Batch size
      - The number of sequences (sentences, documents, etc.) we process in parallel
      - Example: B=32, we're processing 32 sentences at once
    - T = Sequence length
      - The number of tokens per sequence
      - Example: if a sentence has 10 words (tokens), then T = 10
  - Then, $X \in \mathbb{R}^{B \times T \times d_{model}}$
    - The first dimension is the batch
    - The second is the time/sequence
    - The third is the feature/embedding dimension

# Attention in Transformers – Multi-head self-attention

- **Some notes on dimensions**
  - Input: $X \in \mathbb{R}^{B \times T \times d_{model}}$
  - All the heads compute attention in parallel.
    - We concatenate every matrix $W_Q = \left[ W_Q^{(1)} \quad W_Q^{(2)} \quad \dots \quad W_Q^{(h)} \right]$
    - Thus, $W_Q \in \mathbb{R}^{d_{model} \times (h \cdot d_k)}$
    - And, therefore: $Q, K \in \mathbb{R}^{B \times T \times (h \cdot d_k)} \quad V \in \mathbb{R}^{B \times T \times (h \cdot d_v)}$
    - We then reshape Q, K and V intro per-head groups:

$$Q \to (B, h, T, d_k), \quad K \to (B, h, T, d_k), \quad V \to (B, h, T, d_v)$$

      - Now each head works independently

# Attention in Transformers – Multi-head self-attention

- **Some notes on dimensions**
  - After reshaping Q, K and V:

$$Q \to (B, h, T, d_k), \quad K \to (B, h, T, d_k), \quad V \to (B, h, T, d_v)$$

  - We compute Attention
    - For each head $i$: $\text{scores}^{(i)} = \dfrac{Q^{(i)} K^{(i)\top}}{\sqrt{d_k}}$
    - Softmax (across keys): $P^{(i)} = \text{softmax}(\text{scores}^{(i)})$
    - Weighted sum: $O^{(i)} = P^{(i)} V^{(i)}$
      - The shape per head is $(B, T, d_v)$

# Attention in Transformers – Multi-head self-attention

- **Some notes on dimensions**
  - Now, concatenate all heads (along last axis):

$$O = \text{Concat}(O^{(1)}, \ldots, O^{(h)}) \in \mathbb{R}^{B \times T \times (h \cdot d_v)}$$

  - Final projection: $\text{out} = OW_O, \quad W_O \in \mathbb{R}^{hd_v \times d_{model}}$

  - This ensures that the output and the input have the same shapes

output shape = input shape: $(B, T, d_{model})$

# Attention in Transformers – Multi-head self-attention

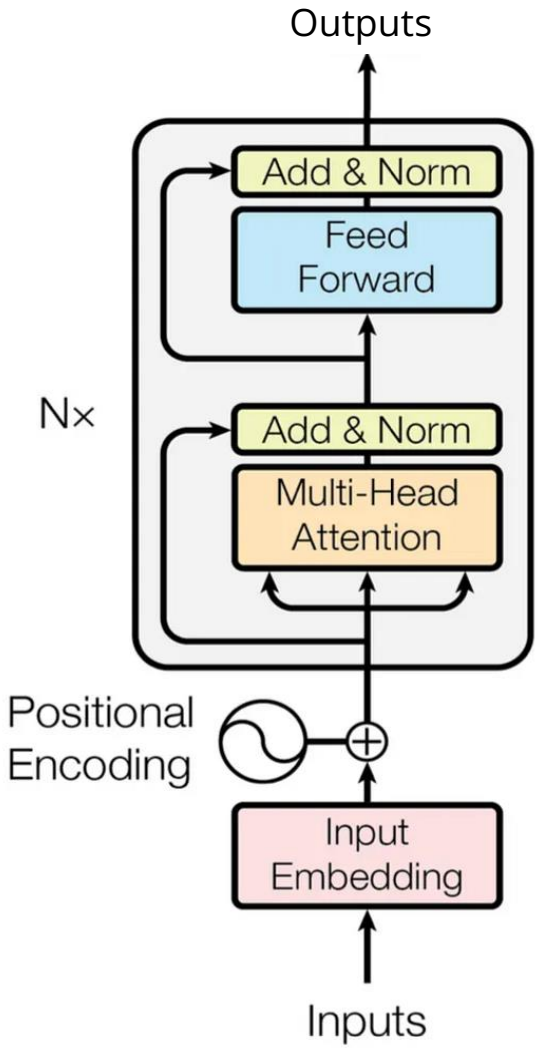
- **Summary**
  - Multi-head attention projects the same sequence into multiple query/key/value subspaces
  - Computes attention in parallel per head
    - In practice, it is also one big matrix multiplication
  - Multiple attention “views” = richer contextual representations
    - Many “lenses” on the same sentence
  - Heads learn different patterns (syntax, semantics, positions…)
  - Concatenation + matrix $W_O$ mixes all perspectives back into one space

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
  - General description
  - Positional encoding
  - Residual connections + layer norm (Add & norm)
  - Feedforward network
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

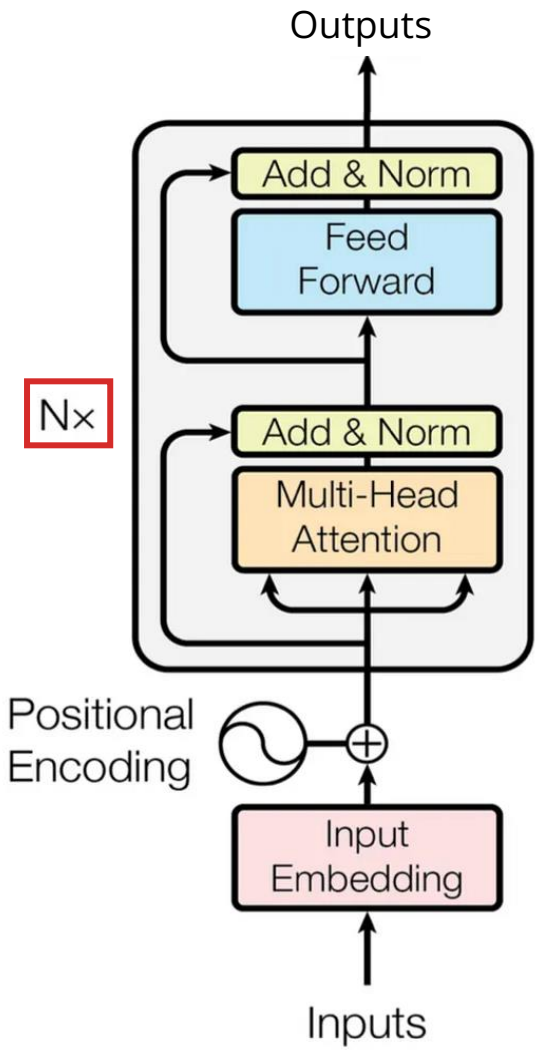
# Transformer architecture: Encoder – General description

- **General overview of the Transformer Encoder**
  - The encoder is the first half of the transformer
    - The other half is the decoder
  - It maps an input sequence into a contextualized representation of the same length
  - **Idea**: Encoder = *stack of attention* + *feedforward* + *normalization layers*



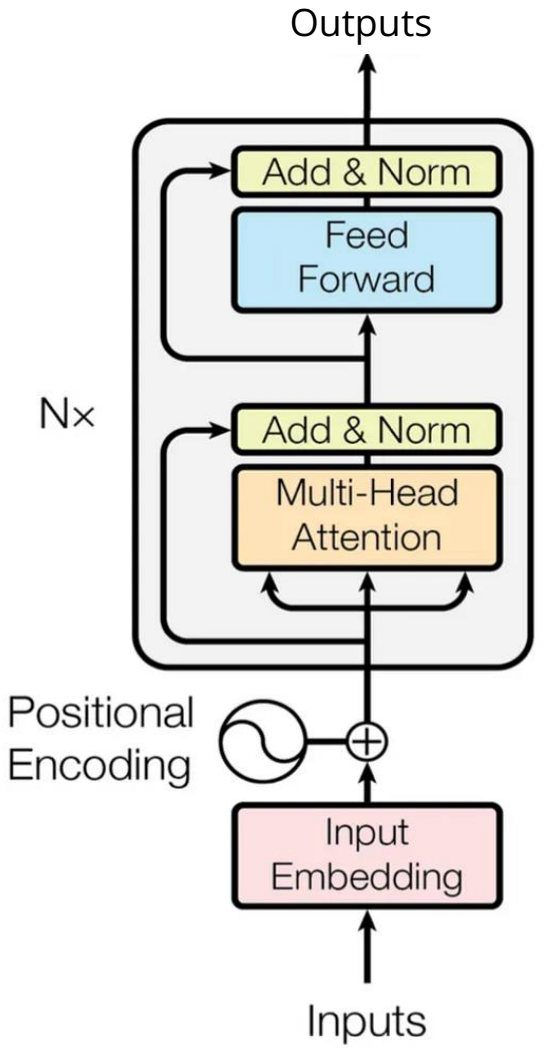
# Transformer architecture: Encoder – General description

- **General overview of the Transformer Encoder**
  - The encoder consists of *N* identical layers
    - Each preserves sequence length *T* and embedding size $d_{model}$
    - *N* = *6* in the original paper
  - Layers share architecture, not weights
    - Each layer has its own parameters
  - General input: $X \in \mathbb{R}^{B \times T \times d_{model}}$
  - General output: Same shape
  - Each output vector contains information about the whole sentence, not just local word



# Transformer architecture: Encoder – General description

- **How the Transformer encoder processes a sequence**
  - Token embeddings
    - Each token is mapped to a vector of size $d_{model}$
    - This embedding matrix is learned during training
  - Positional encoding
    - Added once to the embeddings to inject order info.
  - Stack of *N* encoder layers
    - Each layer:
      - Multi-head self-attention
      - Add + Norm
      - Feedforward network (applied position-wise)
      - Add + Norm
    - Output dimension remains $(B, T, d_{model})$

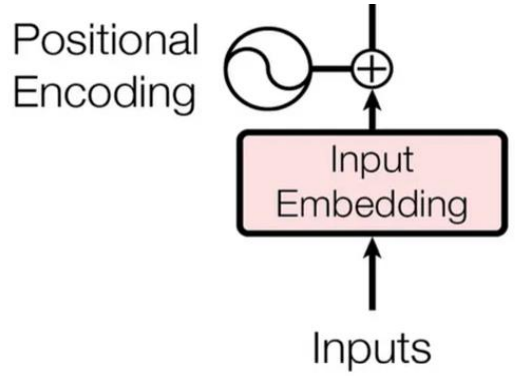


# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
  - General description
  - Positional encoding
  - Residual connections + layer norm (Add & norm)
  - Feedforward network
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

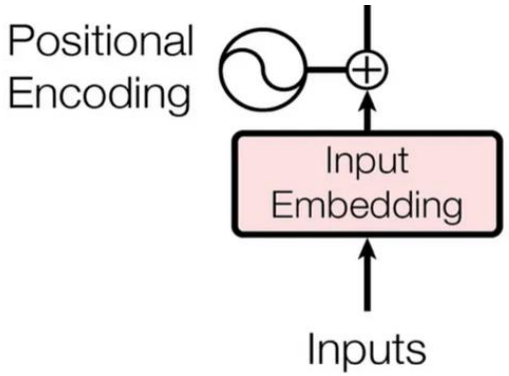
# Transformer architecture: Encoder – Positional encoding

- **Why positional encoding**
  - There is no information about the position of words in a sentence
    - No recurrence (RNN) or convolution (CNN)
  - Attention treats all token as a set (orderless)
    - *A dog bit a man*
    - *A man bit a dog*
  - Without position, self-attention mixes tokens without knowing the order
  - We must inject information about the position explicitly



# Transformer architecture: Encoder – Positional encoding

- **The idea of positional encoding**
  - We add a position vector to token embeddings
    - $\boldsymbol{z}_i = \boldsymbol{x}_i + \boldsymbol{p}_{pos}$
    - This is done once at the input of the encoder
    - Then it is carried through all layers




  - **Options**:
    - Fixed sinusoidal encoding (original Transformer).
      - Deterministic, no extra parameters
    - Learned positional embeddings (later madels, e.g. BERT)
      - Trained like word embeddings

# Transformer architecture: Encoder – Positional encoding

- **Sinusoidal encoding**
  - We are looking for a vector $\boldsymbol{p}_{pos}$ that encodes the position of a token
  - The definition is done component-wise:
    - For dimension index $i \in \{0,1,\ldots,\frac{d_{model}}{2}-1\}$:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

      - Different frequencies across dimensions
      - Provides unique encoding for each position
      - Relative positions can be derived

# Transformer architecture: Encoder – Positional encoding

- **Sinusoidal encoding**

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$\boldsymbol{p}_{pos} = [p_1, p_2, p_3, p_4, \dots, p_{d_{model}}]$$

$i = 0$ $i = 1$ ...

# Transformer architecture: Encoder – Positional encoding

- **Sinusoidal encoding**
  - Positional encoding vector $\boldsymbol{p}_{pos}$

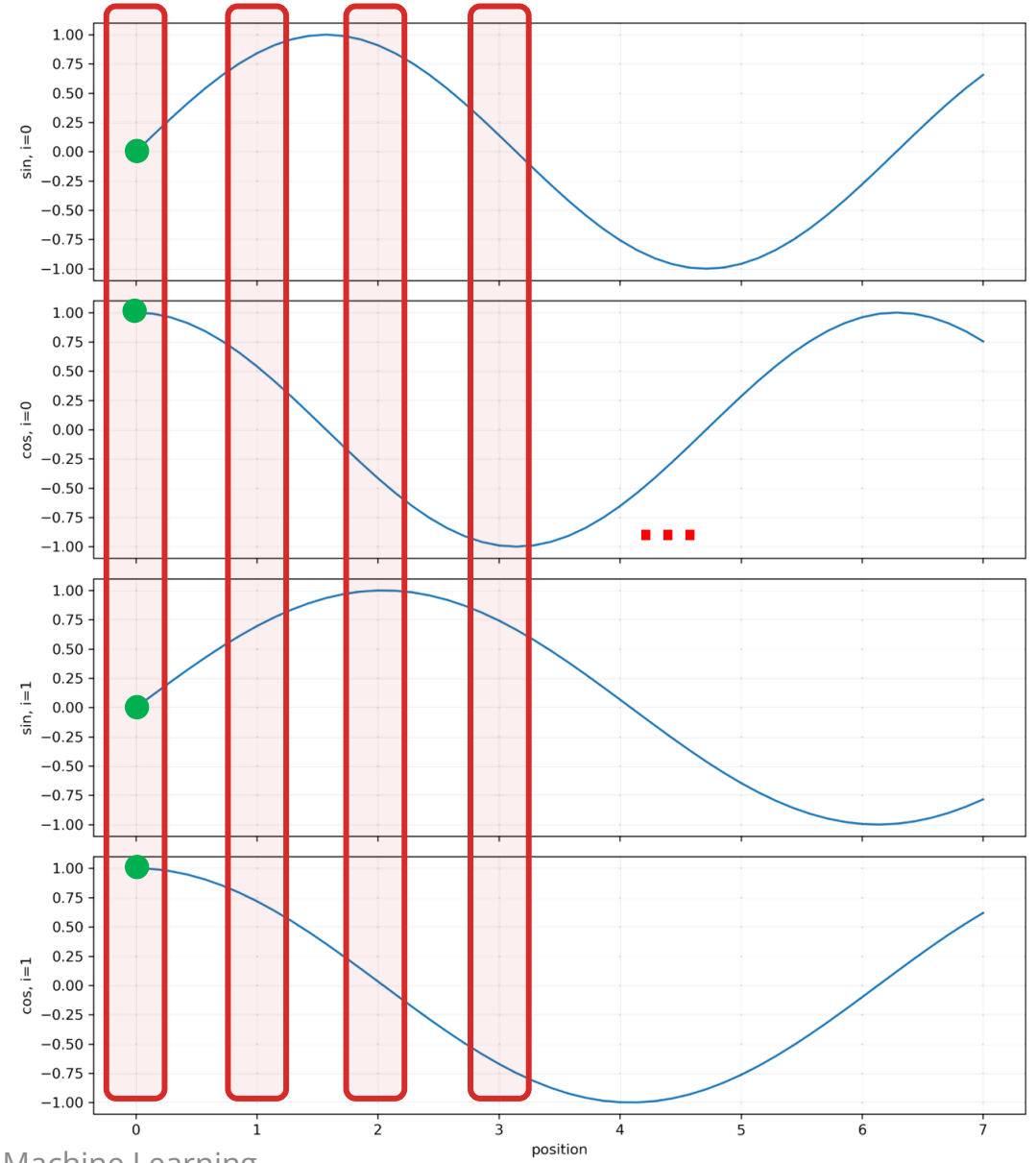
$$\boldsymbol{p}_0 = [p_{0,0}, p_{0,1}, p_{0,2}, p_{0,3}, \dots]$$

$$\boldsymbol{p}_1 = [p_{1,0}, p_{1,1}, p_{1,2}, p_{1,3}, \dots]$$

$$\boldsymbol{p}_2 = [p_{2,0}, p_{2,1}, p_{2,2}, p_{2,3}, \dots]$$

$$\boldsymbol{p}_3 = [p_{3,0}, p_{3,1}, p_{3,2}, p_{3,3}, \dots]$$

$$\vdots$$



# Transformer architecture: Encoder – Positional encoding

- **Learned positional encoding**
  - Alternative to sinusoidal encoding from the original Transformer
    - Used in BERT, GPT-2, GPT-3, etc.
  - The idea is to let the model learn a vector for each position index

$$\boldsymbol{p}_{pos} \in \mathbb{R}^{d_{model}}$$

  - Therefore, we create a lookup table like the word embedding table

$$\boldsymbol{P} \in \mathbb{R}^{T_{max} \times d_{model}}$$ $T_{max}$ is the maximum sequence length

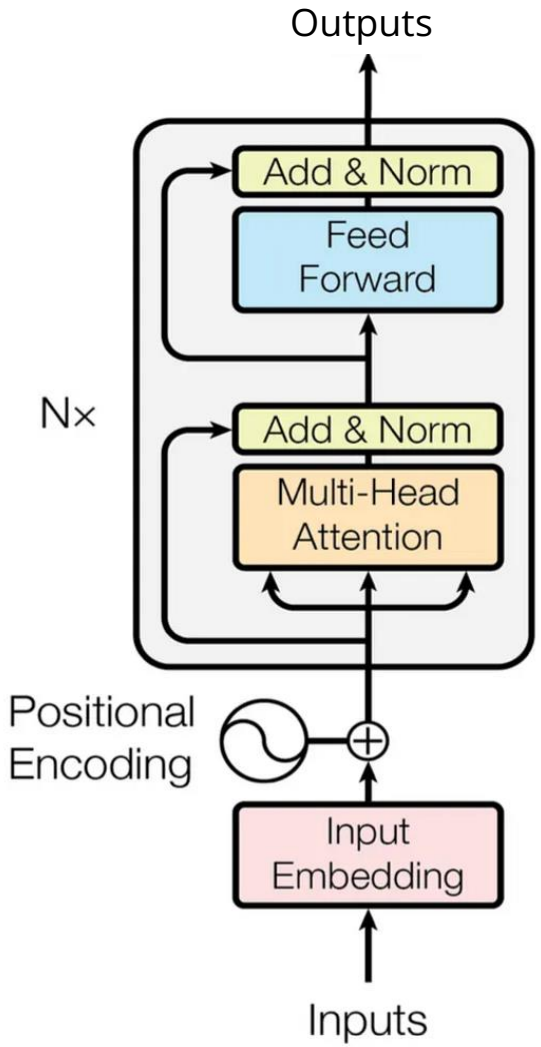
  - And, at runtime, $z_i = x_i + P_{pos(i)}$
    - *P* are trainable parameters optimized with the rest of the model
      - They could learn anything, but in practice they correlate with positions

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
  - General description
  - Positional encoding
  - Residual connections + layer norm (Add & norm)
  - Feedforward network
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Transformer architecture: Encoder – Add & Norm

- **Residual connections and layer normalization**
  - Key components that make deep Transformers trainable and stable
  - Used around every sublayer (attention and feedforward)
  - Inspired by ResNets (He et al., 2015)
    - https://arxiv.org/pdf/1512.03385
  - Avoid gradients vanishing or exploding
  - Keep the flow of information stable



# Transformer architecture: Encoder – Add & Norm

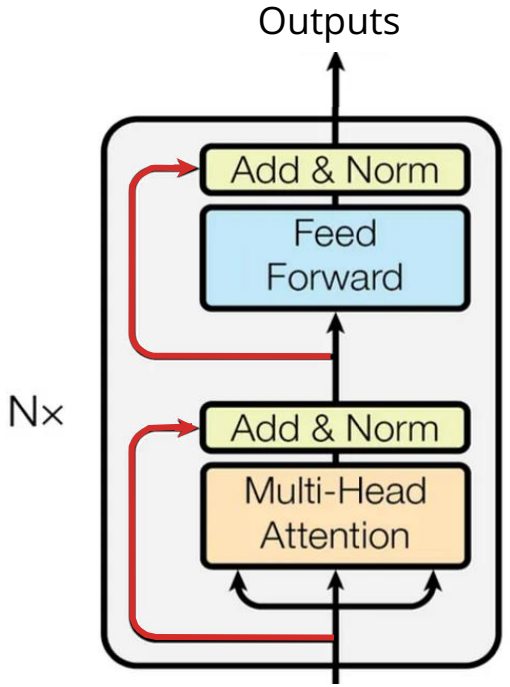
- **Residual connection**
  - A residual connection adds the layer's input to its output

$$\text{Output} = x + \text{Sublayer}(x)$$

    - $x$ is the input of the sublayer
    - *Sublayer(x)* is the output of a multi-head attention or feedforward network

  - It requires the same shape $(B, T, d_{model})$ for addition

  - If the transformation is small, the model preserves the input
  - If the transformation is useful, residuals add it on top
  - The idea is that each layer learns a refinement, rather than a complete rewrite



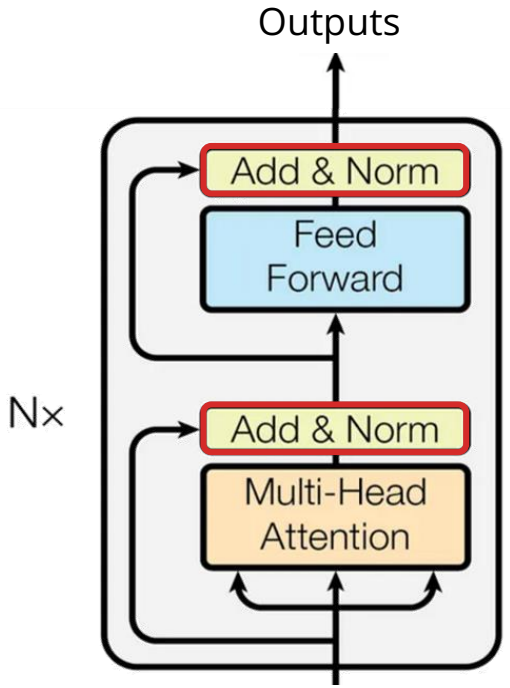
# Transformer architecture: Encoder – Add & Norm

- **Layer normalization**
  - Normalizes across the feature dimension for each token

$$\mathrm{LN}(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

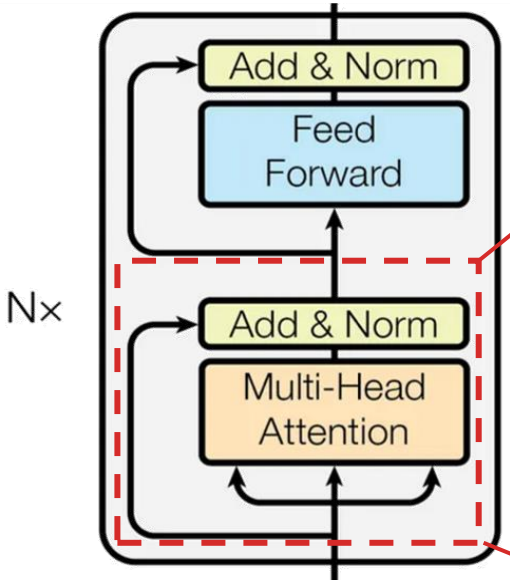
    where
    - $\mu$ and $\sigma$ are the mean and standard deviation across the $d_{model}$ dimensions of one token's vector
    - $\gamma$ and $\beta$ are learnable scaling and shifting parameters
  - Normalizes **per token** (not per Batch, as in BatchNorm)

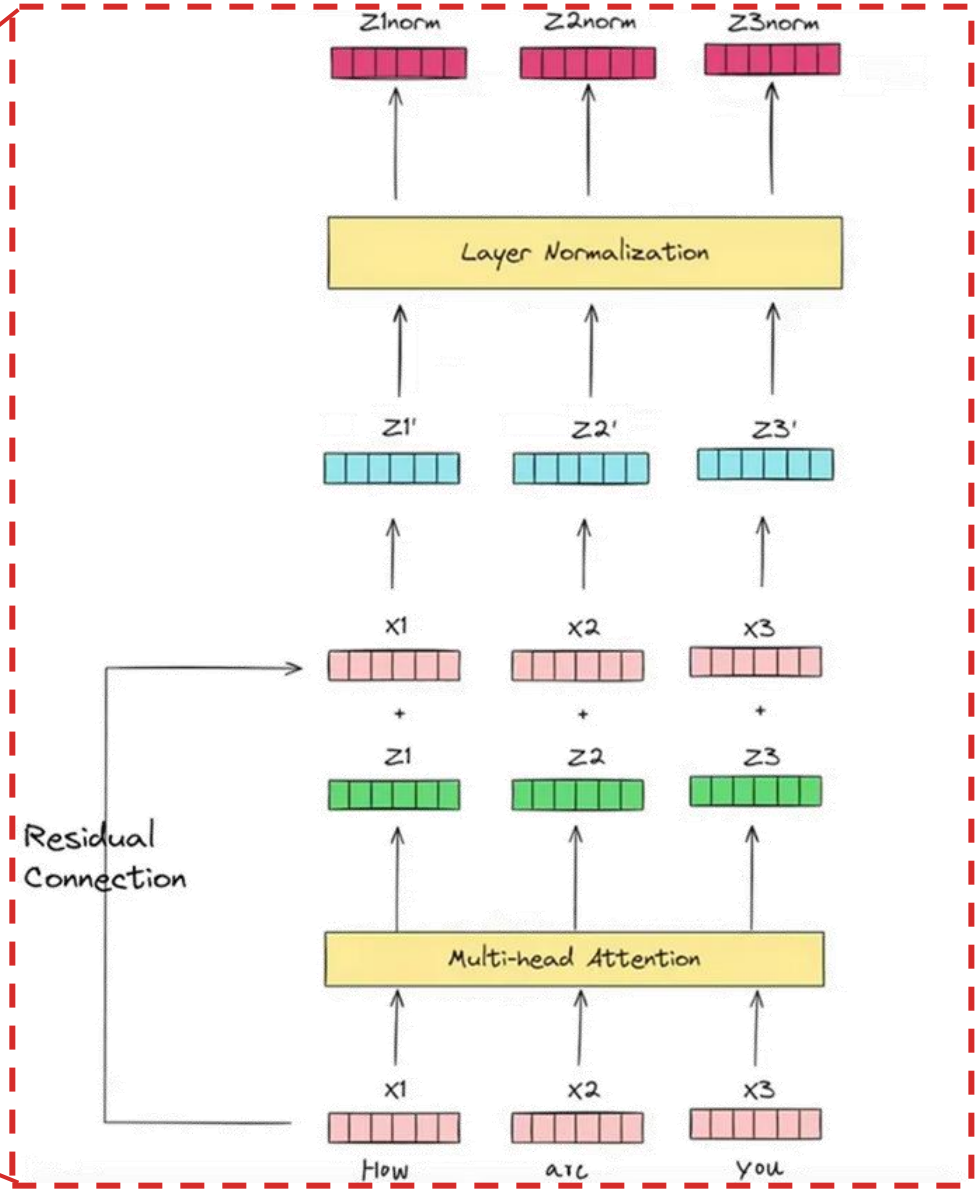
  - Keeps activations within a stable range
  - Works better than BatchNorm for sequential data
  - Helps gradients flow evenly through very deep stacks



# Transformer architecture: Encoder – Add & Norm

- **Visual representation of Add & Norm**



# Transformer architecture: Encoder – Add & Norm

- **Order variants**
  - Most modern Transformers differ in the order of operations
    - Original transformer

$$\mathrm{LN}(x + \mathrm{Sublayer}(x))$$

    - Modern transformers (BERT, GPT)

$$x + \mathrm{Sublayer}(\mathrm{LN}(x))$$

      - Improves gradient stability for very deep models

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
  - General description
  - Positional encoding
  - Residual connections + layer norm (Add & norm)
  - Feedforward network
- Transformer architecture: Decoder
- Training and inference tips
- Other architectures

# Transformer architecture: Encoder – Feedforward network

- **Feedforward network**
  - The second sublayer in every decoder block
  - Applied after multi-head attention and Add&Norm
  - Transforms each token's embedding independently
    - Refining individual representation

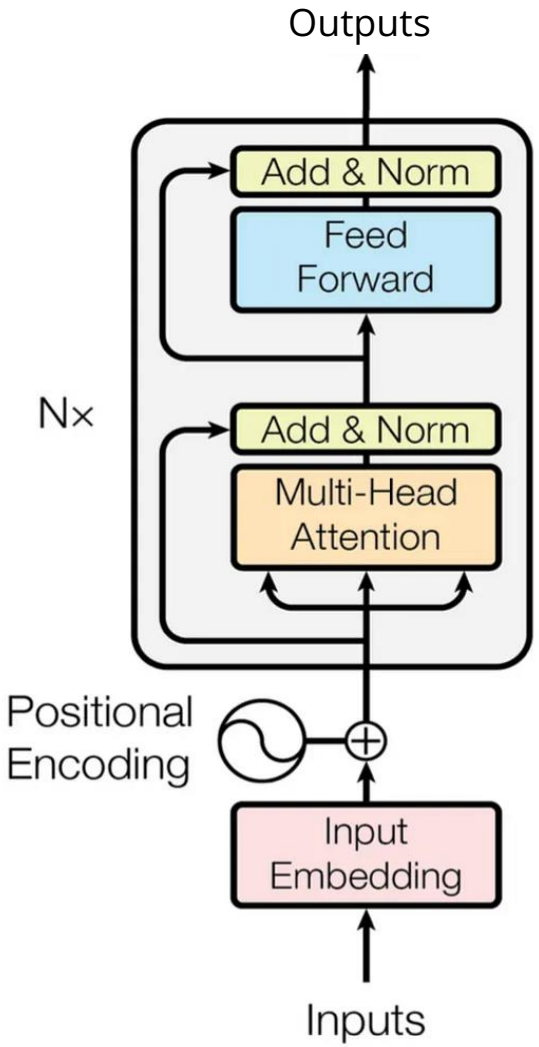
$$\mathrm{FFN}(X) = \mathrm{Activation}(XW_1 + b_1)W_2 + b_2$$

$$X \in \mathbb{R}^{B \times T \times d_{model}}$$

$$W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$$

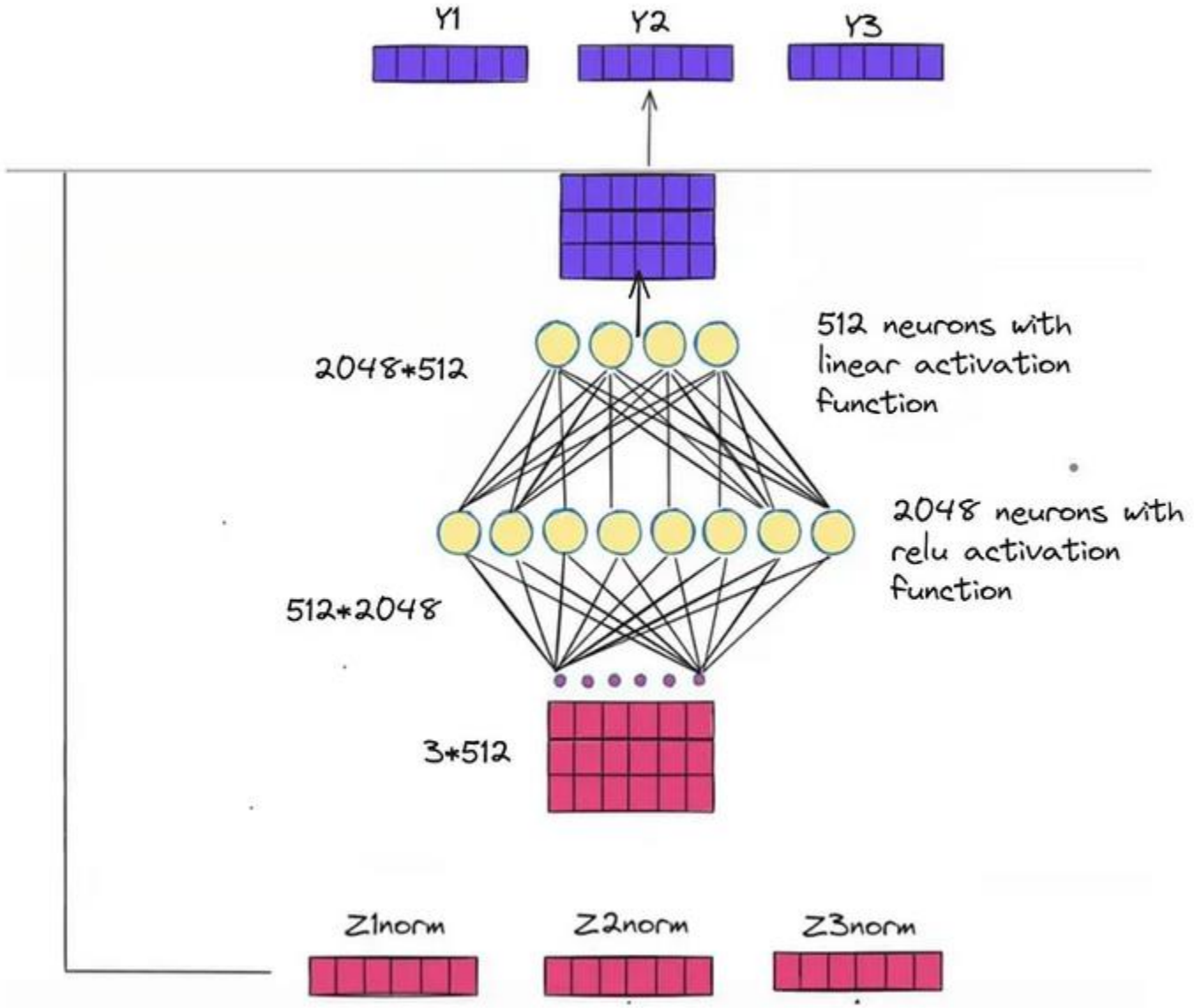
$$W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$$

$b_1, b_2$ are bias vectors.



# Transformer architecture: Encoder – Feedforward network

- **Dimension expansion**
  - The FFN increases the hidden dimension to $d_{ff}$
    - Often 4 times larger
  - Gives the model more nonlinear capacity per token



# Transformer architecture: Encoder – Feedforward network

- **Activation function**
  - ReLU (Rectified Linear Unit):

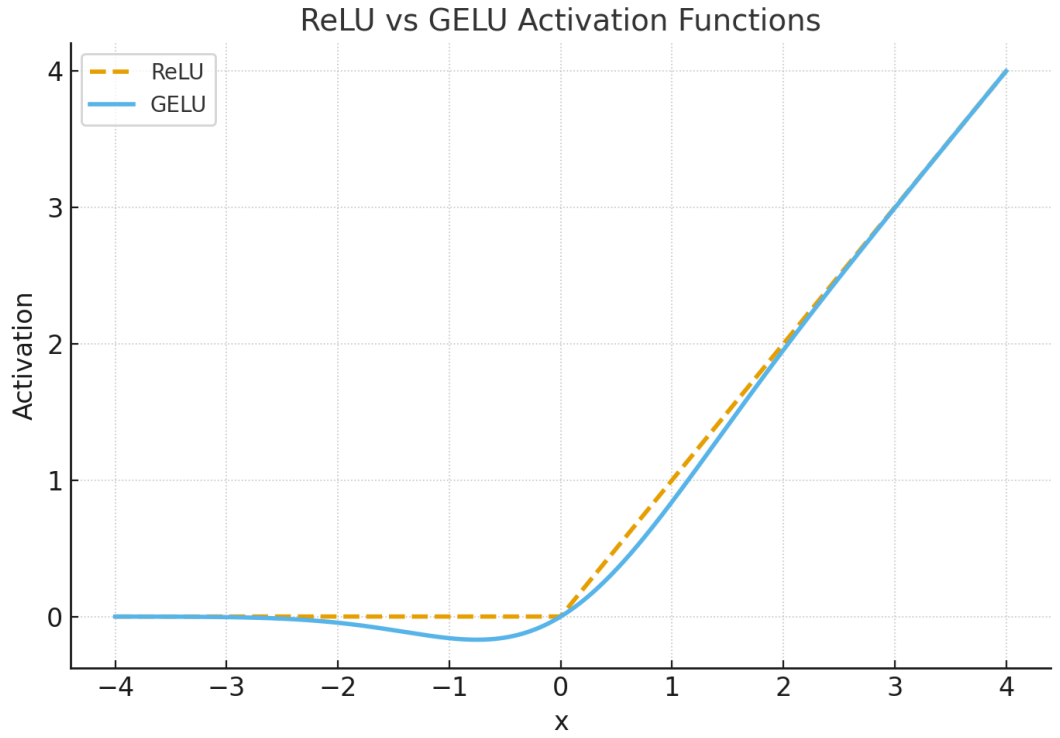
$$\mathrm{ReLU}(x) = \max(0, x)$$

    - Used in original Transformer

  - GELU (Gaussian Error Linear Unit):

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

    - where $\Phi(x)$ is the cumulative Gaussian function of the standard normal distribution
    - Used in modern Transformers (BERT, GPT)
    - Approximation used in practice: $\mathrm{GELU}(x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$

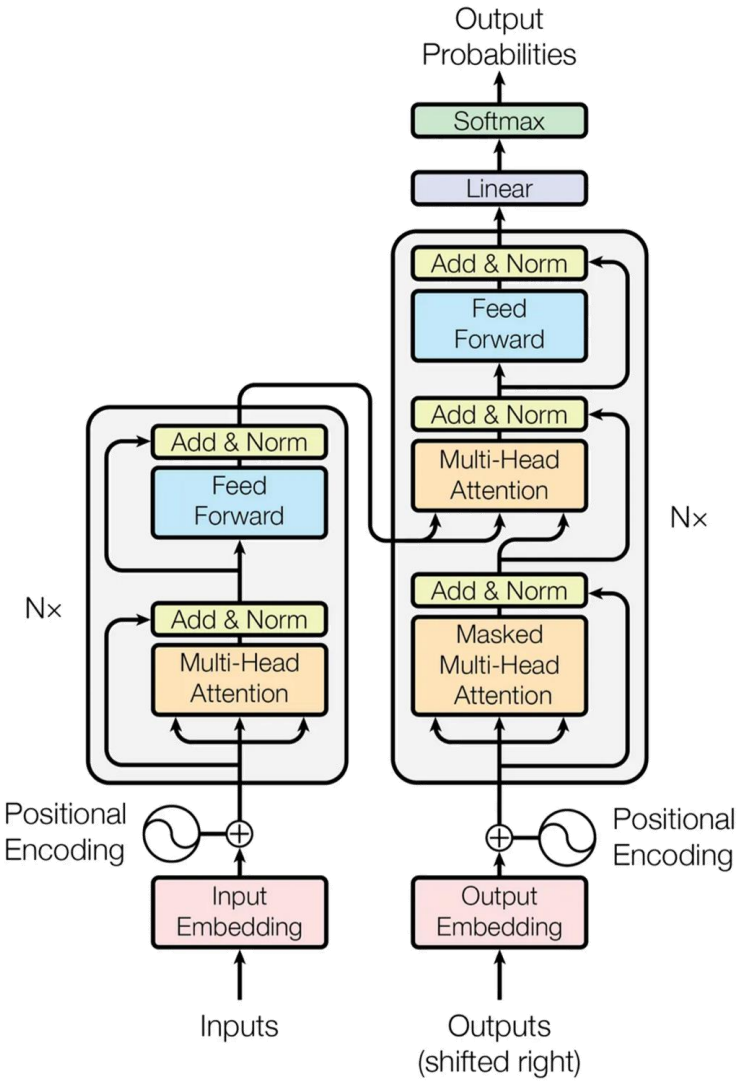


# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Transformer architecture: Decoder – General description

- **Decoder overview**
  - The decoder generates the target sequence one token at a time
  - Each token prediction depends on:
    - All previously generated tokens (masked self-attention)
    - The encoded source sequence (cross attention)
  - It is built from stacked layers combining:
    - Positional encoding
    - Masked multi-head self-attention
    - Cross-attention
    - Feedforward network
    - Add & Norm connections

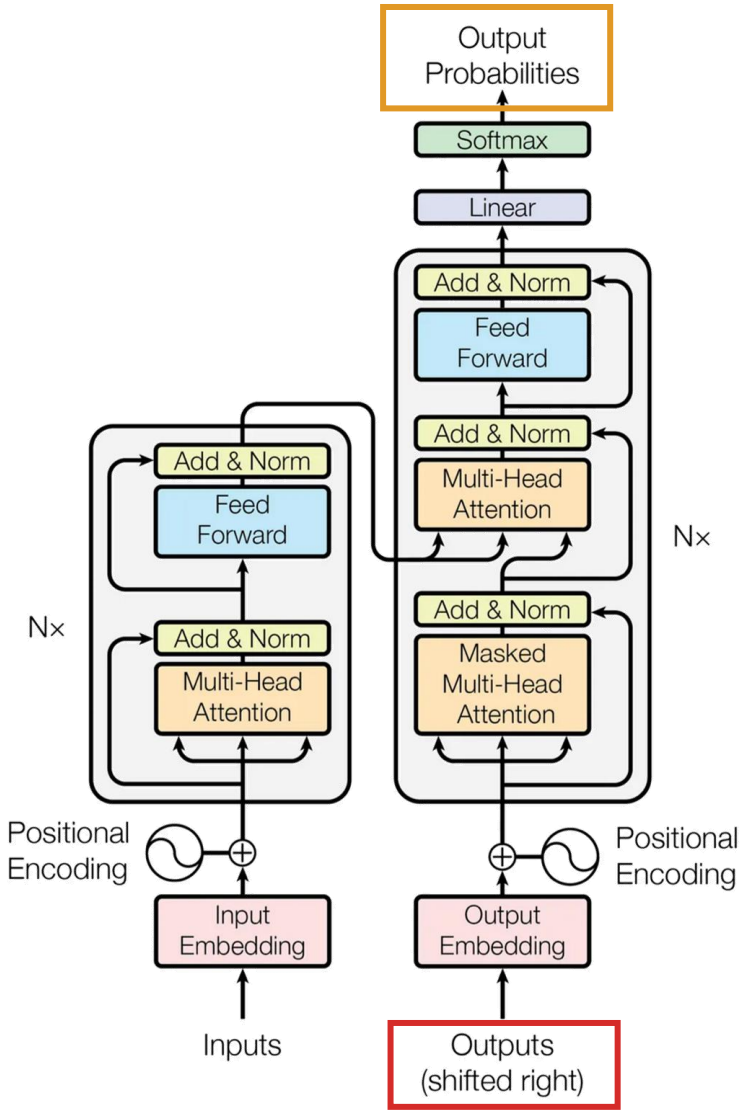


# Transformer architecture: Decoder – General description

- **Decoder overview**
  - Input: previous target tokens (shifted)
    - <BOS> (beginning of sentence) token at start
  - Output: probability distribution for the next token

  - **During training:**
    - Teacher forcing mode

  - **During inference:**
    - Autoregressive

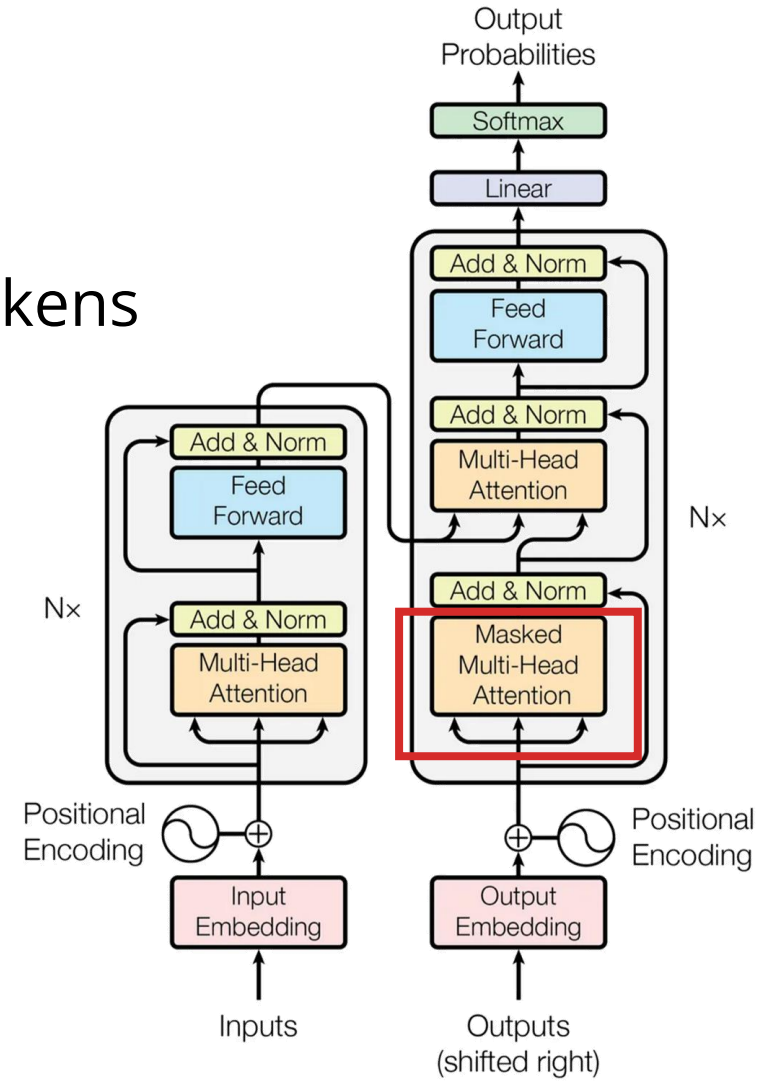


# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Transformer architecture: Decoder – Masked Multi-head att.

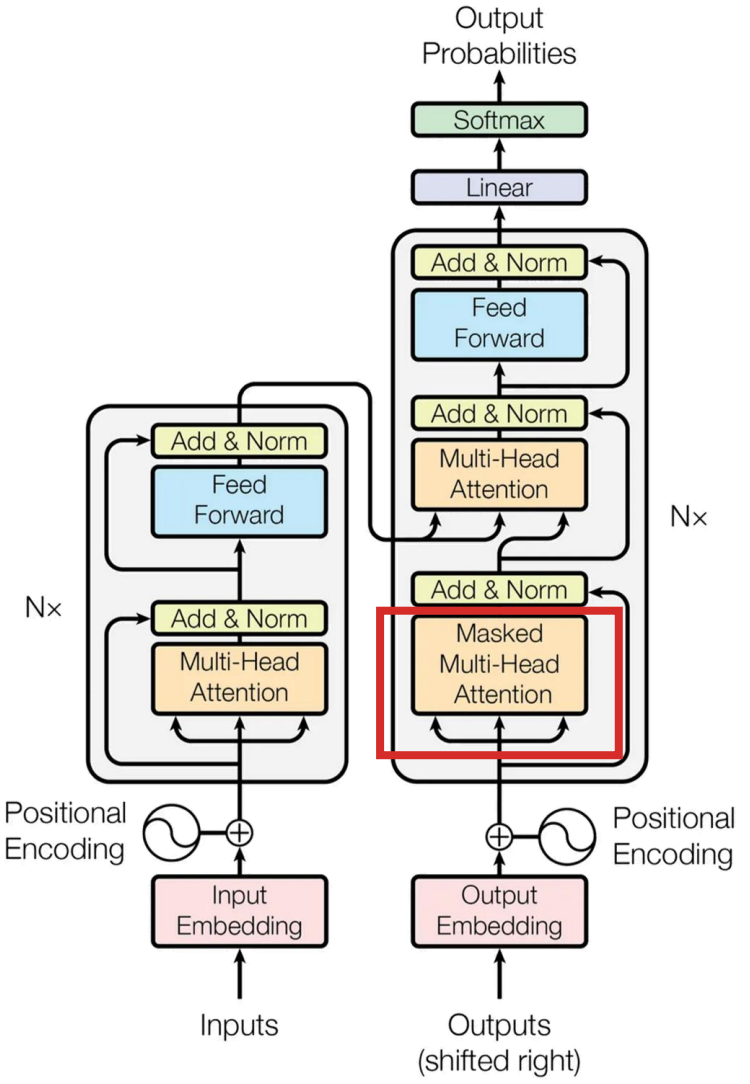
- **Masked multi-head self-attention**
  - The decoder uses masked self-attention
    - To ensure causal dependency
    - Each token only attends to itself and previous tokens
      - No peeking into the future
  - This ensures autoregressive generation
    - $y_t$ only depends on $y_{<t}$
  - Still parallelizable during training (teacher forcing)



# Transformer architecture: Decoder – Masked Multi-head att.

- **Masked multi-head self-attention**
  - Example with sequence
    - [<BOS>, I, like, pizza]
    - At position “like”:
      - Can attend to <BOS>, “I”, and itself
      - Cannot attend to “pizza”
    - [<BOS>, I, like, pizza]

masked



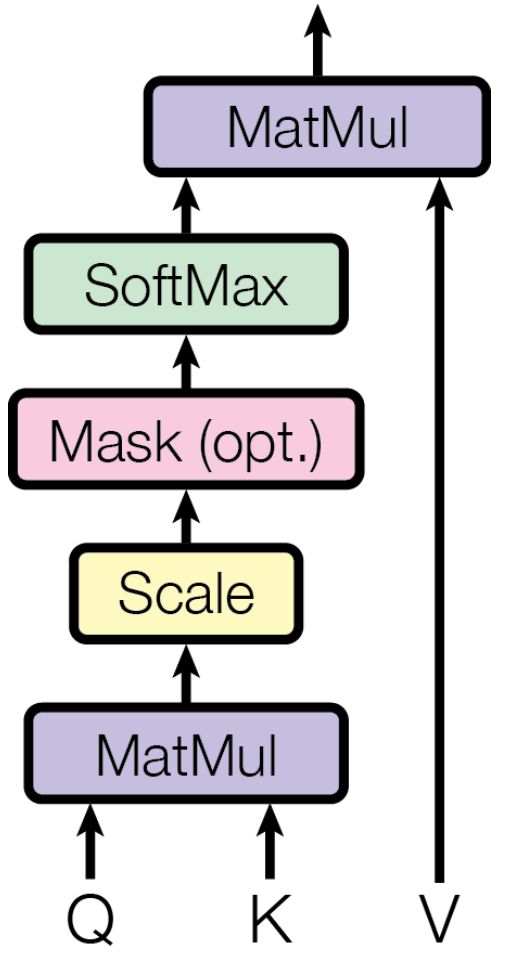
# Transformer architecture: Decoder – Masked Multi-head att.

- **Mathematical formulation**
  - Similar to multi-head self-attention
  1. Compute $Q = XW_Q, \quad K = XW_K, \quad V = XW_V$
  2. Compute the scores

$$\text{scores} = \frac{QK^\top}{\sqrt{d_k}}$$

  3. Mask the upper triangle
     - Set to $-\infty$
  4. Compute attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\text{scores})V$$



# Transformer architecture: Decoder – Masked Multi-head att.



- **Example: "The hungry cat chased the mouse"**
  - Setup
    - $d_{model} = 6$ (one-hot embedding per token)
    - $d_k = d_v = 2$
    - $X$ = the 6x6 identity matrix

$$W_Q = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0.5 & 0 \\ 0 & 1 \end{bmatrix} \quad W_K = \begin{bmatrix} 0.2 & 0 \\ 0.6 & 0 \\ 1.2 & 0 \\ 0 & 1 \\ 0.2 & 0 \\ 0.8 & 0 \end{bmatrix} \quad W_V = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

# Transformer architecture: Decoder – Masked Multi-head att.



- **Example: "The hungry cat chased the mouse"**
  - Compute $Q = XW_Q, \quad K = XW_K, \quad V = XW_V$

$$Q = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0.5 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 0.2 & 0 \\ 0.6 & 0 \\ 1.2 & 0 \\ 0 & 1 \\ 0.2 & 0 \\ 0.8 & 0 \end{bmatrix} \quad V = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

# Transformer architecture: Decoder – Masked Multi-head att.



- **Example: “The hungry cat chased the mouse”**
  - Scores

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad \sqrt{d_k} = \sqrt{2} \approx 1.414$$

$$S \approx \begin{bmatrix} 0.071 & 0.212 & 0.424 & 0 & 0.071 & 0.283 \\ 0.141 & 0.424 & 0.849 & 0 & 0.141 & 0.566 \\ 0 & 0 & 0 & 0.707 & 0 & 0 \\ 0.141 & 0.424 & 0.849 & 0 & 0.141 & 0.566 \\ 0.071 & 0.212 & 0.424 & 0 & 0.071 & 0.283 \\ 0 & 0 & 0 & 0.707 & 0 & 0 \end{bmatrix}$$

# Transformer architecture: Decoder – Masked Multi-head att.



- **Example: “The hungry cat chased the mouse”**
  - Compute the masked scores matrix

$$S' = \begin{bmatrix} 0.071 & -\infty & -\infty & -\infty & -\infty & -\infty \\ 0.141 & 0.424 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty & -\infty \\ 0.141 & 0.424 & 0.849 & 0 & -\infty & -\infty \\ 0.071 & 0.212 & 0.424 & 0 & 0.071 & -\infty \\ 0 & 0 & 0 & 0.707 & 0 & 0 \end{bmatrix}$$

# Transformer architecture: Decoder – Masked Multi-head att.



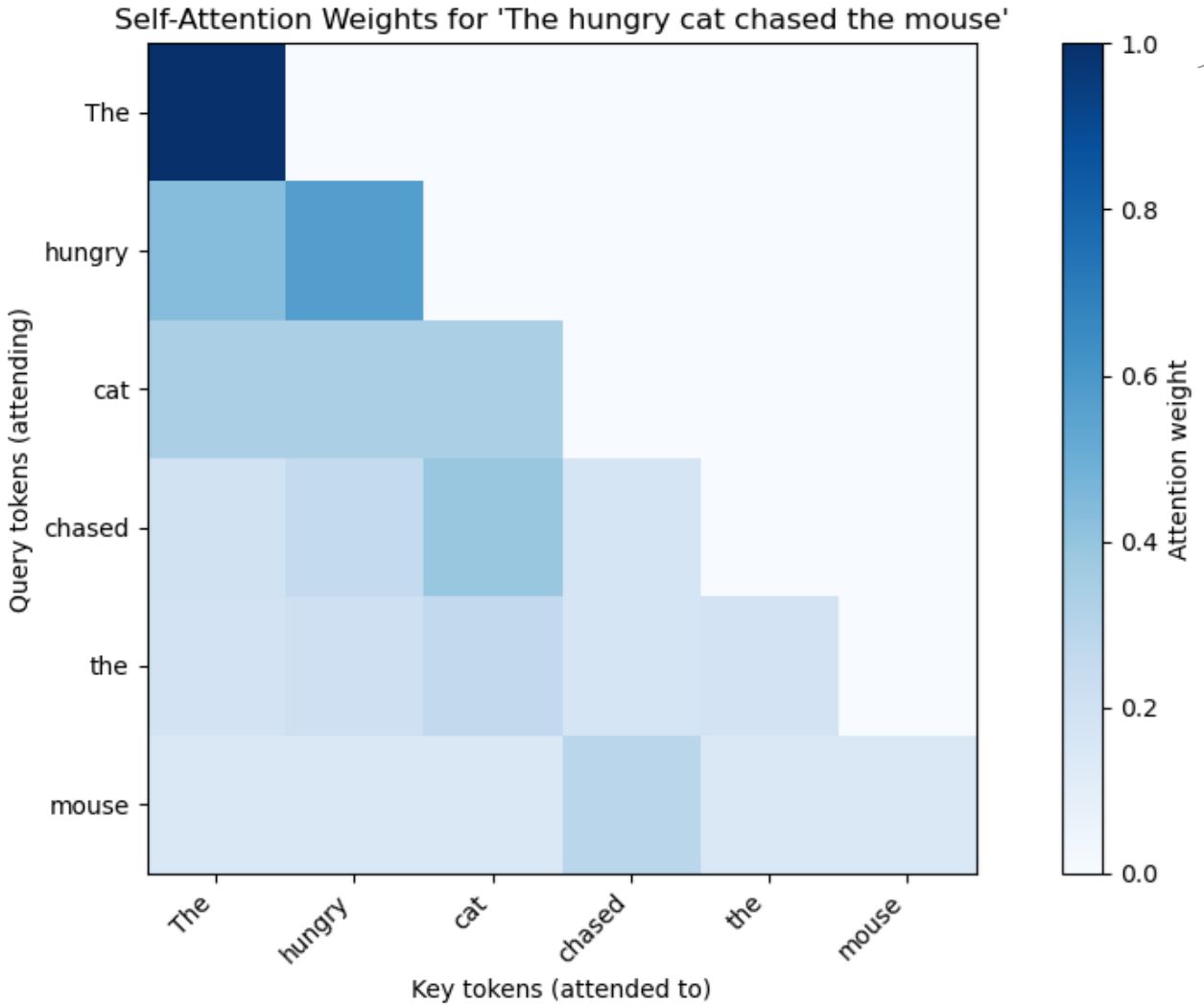
- **Example: “The hungry cat chased the mouse”**
  - Softmax (row-wise) – over unmasked values

$$A \approx \begin{bmatrix} 1.000 & 0 & 0 & 0 & 0 & 0 \\ 0.426 & 0.574 & 0 & 0 & 0 & 0 \\ 0.333 & 0.333 & 0.333 & 0 & 0 & 0 \\ 0.201 & 0.267 & 0.359 & 0.173 & 0 & 0 \\ 0.196 & 0.226 & 0.279 & 0.168 & 0.131 & 0 \\ 0.206 & 0.206 & 0.206 & 0.239 & 0.143 & 0 \end{bmatrix}$$

# Transformer architecture: Decoder – Masked Multi-head att.



- **Example: “The hungry cat chased the mouse”**



# Transformer architecture: Decoder – Masked Multi-head att.

- **Multi-head masked attention**
  - Each head applies the same mask independently
    - Each heads captures different dependencies, but within the causal limit
  - This is still fully parallelizable during training
  - Each head has a distinct subspace view, but all obey the mask

| Property | Encoder Self-Attention | Decoder Masked Self-Attention |
|---|---|---|
| Visibility | All tokens see each other | Only past tokens visible |
| Parallelizable? | Fully parallel | Still parallel (during training) |
| Causality | Non-causal | Causal (autoregressive) |

# Transformer architecture: Decoder – Masked Multi-head att.

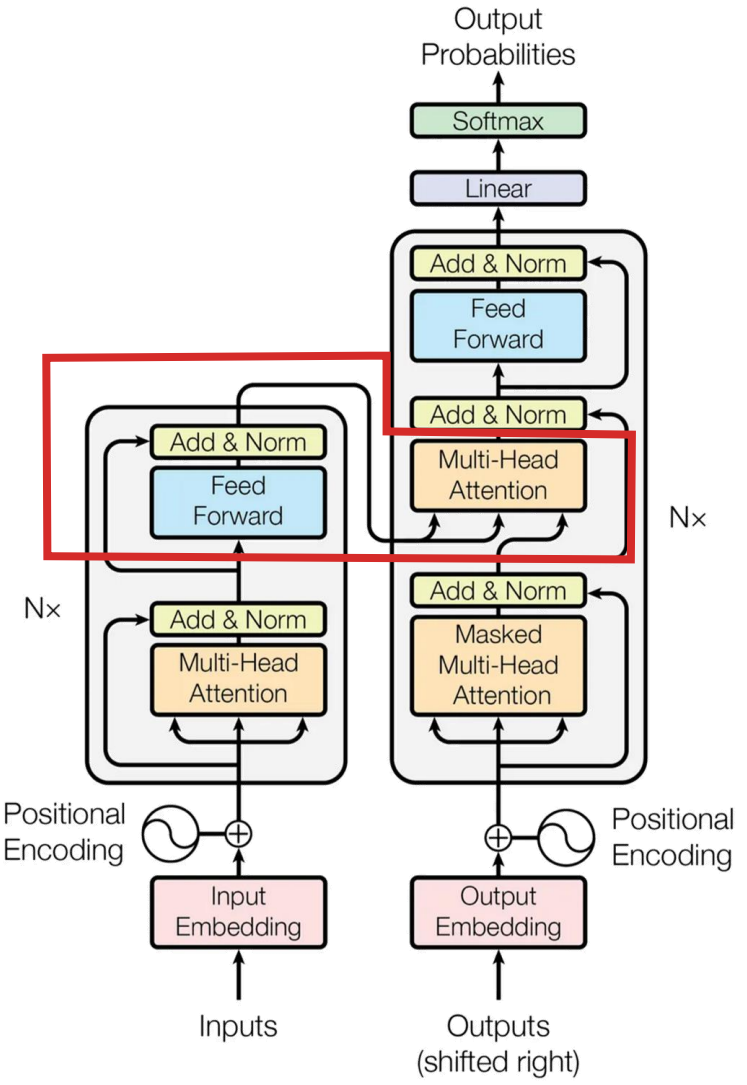
- **Multi-head masked attention: training vs inference**
  - During training
    - The full target sentence is available: $y = [<\text{BOS}>, y_1, y_2, \ldots, y_T]$
    - All tokens are processed in parallel: the model computes attention for every position simultaneously
    - The mask prevents each token from "seeing" future tokens
      - Enforces causality
      - Allows parallel training without breaking the autoregressive rue
  - During inference
    - The model only knows previously generated tokens
    - Generates one token at a time (masking is implicit)
    - Computation is sequential, not parallel

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Transformer architecture: Decoder – Cross-attention

- **Cross-attention: Connecting encoder and decoder**
  - The decoder doesn't only rely on its own past outputs
  - It must attend to the encoder's output
    - Cross-attention (a.k.a. encoder-decoder attention)
  - After mask self-attention, each decoder position holds context over previous target tokens
  - To predict the next token, it also needs to know which parts of the source are relevant



# Transformer architecture: Decoder – Cross-attention

- **Mathematical formulation**
  - It is a standard multi-head attention, but
    - Q comes from the decoder
    - K and V come from the encoder

    - Encoder output: $H_{\text{enc}} \in \mathbb{R}^{T_x \times d_{\text{model}}}$
    - Decoder hidden state: $X_{\text{dec}} \in \mathbb{R}^{T_y \times d_{\text{model}}}$

$$Q = X_{\text{dec}} W_Q, \quad K = H_{\text{enc}} W_K, \quad V = H_{\text{enc}} W_V$$

$$\Rightarrow \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \in \mathbb{R}^{T_y \times d_v}$$

# Transformer architecture: Decoder – Cross-attention

- **Mathematical formulation**
  - The rest is as in multi-head self-attention:
  - Outputs are concatenated and linearly projected via $W_o$

- **Example**
  - Source: “I like apples”
  - Target: “<BOS> Me gustan las manzanas”

  - At step “pizza”:
    - Decoder query (“manzanas”) attends mostly to encoder token “apple”

# Transformer architecture: Decoder – Cross-attention

- **Connection to classical seq2seq attention**
  - Cross-attention generalizes the Bahdanau attention in RNNs
  - But now:
    - Multi-headed
    - Fully parallel
    - Learned via matrix operations instead of time-step loops

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Transformer architecture: Decoder – Last layers

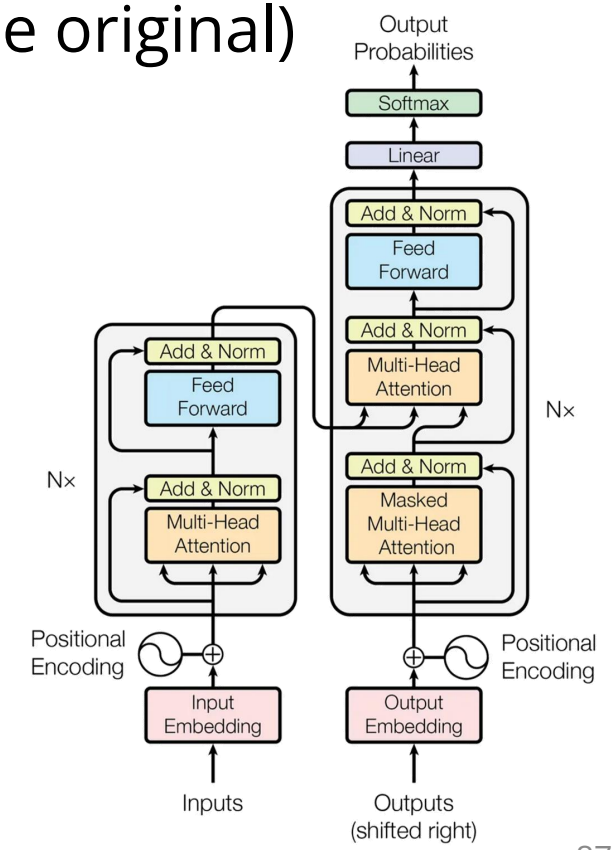
- **Decoder**

Input + positional encoding → Layer 1 → Layer 2 → Layer 3 → … → Layer N → Linear + Softmax

  - The decoder is composed of N identical layers (N=6 in the original)

1. Masked multi-head self-attention
2. Cross-attention (encoder-decoder)
3. Feedforward network
4. Add & Norm operations after each sublayer

  - The output of one layer becomes the input to the next
  - Each layer has its own parameters
    - Nothing is shared between layers



# Transformer architecture: Decoder – Last layers

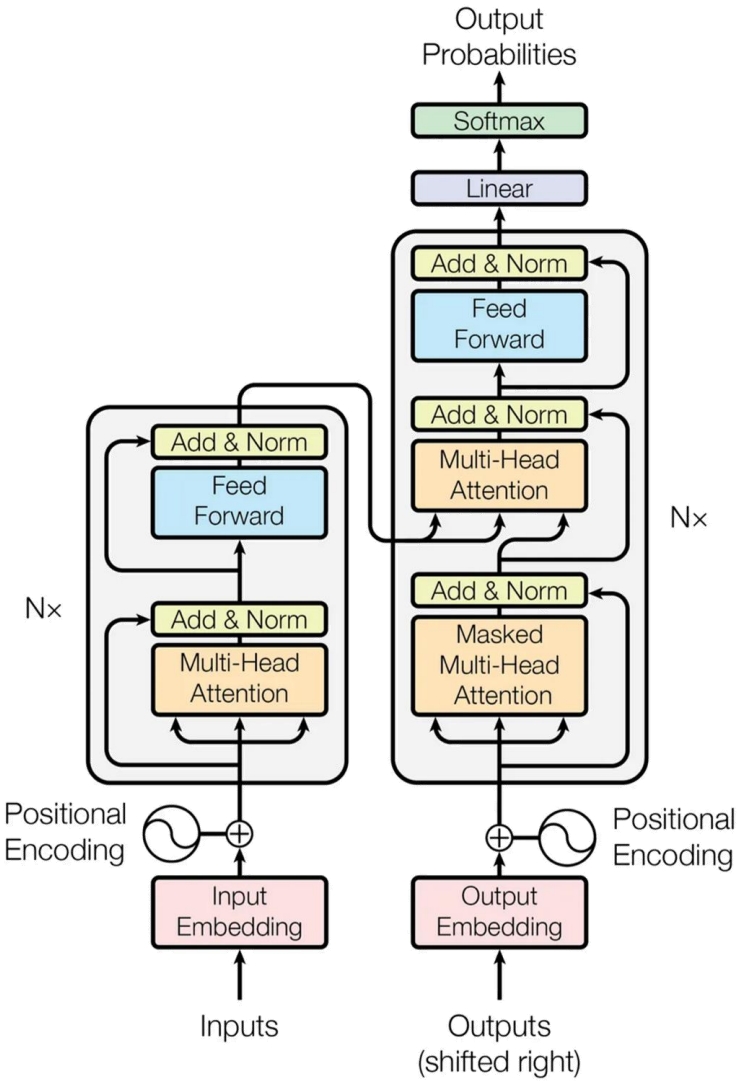
- **Add & Norm in the decoder**
  - After every sublayer, we apply residual connection and layer normalization

$$\text{Output} = \text{LayerNorm}(X + \text{Sublayer}(X))$$

    - The same principle used in the encoder
- **Feedforward network in the decoder**
  - Each position's vector is passed through a FFN

$$\text{FFN}(X) = \text{Activation}(XW_1 + b_1)W_2 + b_2$$

  - Same as in the encoder
    - First layer expands dimensionality
    - Second layer projects back to $d_{model}$

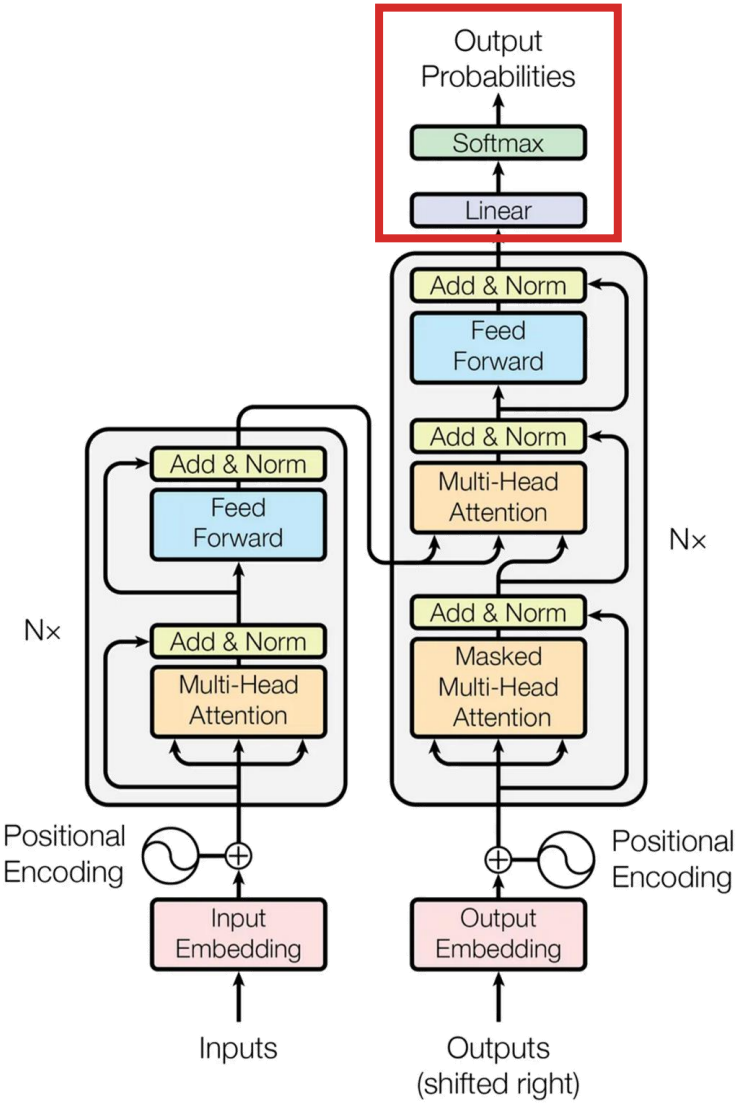


# Transformer architecture: Decoder – Last layers

- **From hidden states to token probabilites**
  - The decoder's last layer outputs contextual representations for each target token:

$$H_{\text{dec}} \in \mathbb{R}^{(B,T_y,d_{\text{model}})}$$

  - These must be converted into probabilities over the vocabulary
    - We want $P(y_t|y_{<t}, x)$
  - To do this:
    - Linear layer projecting to vocabulary size
    - Softmax layer converting logits to probabilities



# Transformer architecture: Decoder – Last layers

- **Linear projection step**
  - Input: $H_{\text{dec}} \in \mathbb{R}^{(B, T_y, d_{\text{model}})}$
  - Linear layer:

$$\text{logits} = H_{\text{dec}} W_{\text{vocab}}^T + b$$

$$H_{\text{dec}} : (B, T_y, d_{\text{model}})$$

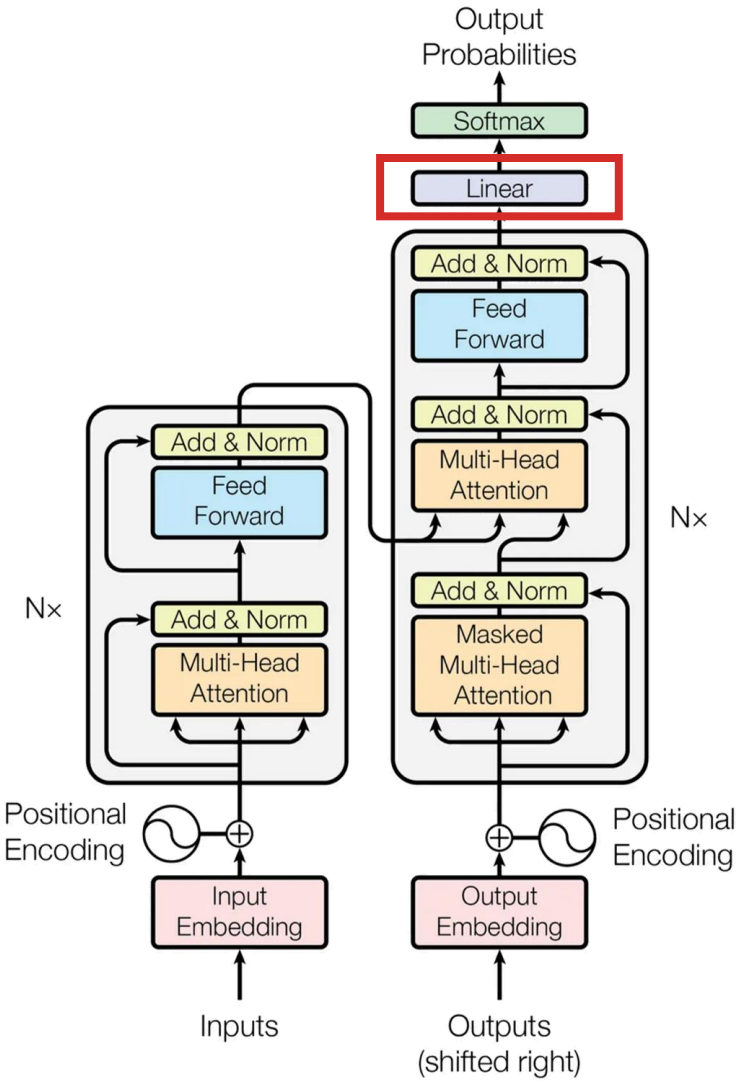
$$W_{\text{vocab}} : (V, d_{\text{model}})$$

$$b : (V,)$$

  - Output:

$$\text{logits} \in \mathbb{R}^{(B, T_y, V)}$$

**Each number**: model's unnormalized belief that this word should be next

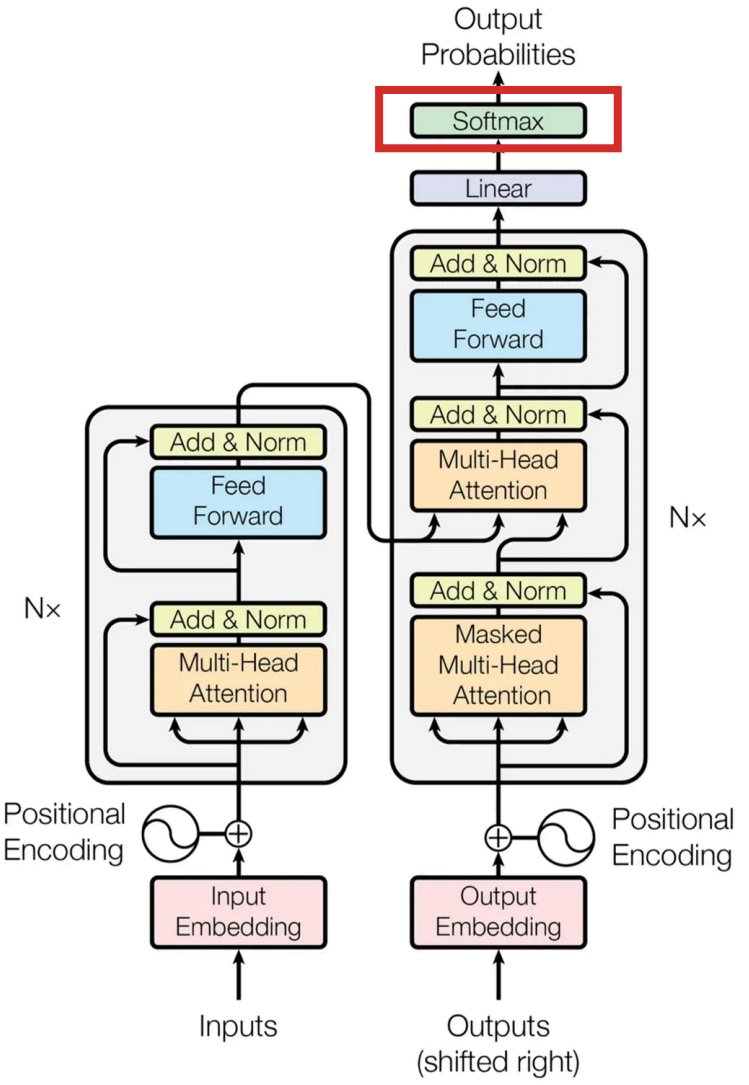


# Transformer architecture: Decoder – Last layers

- **Softmax output layer**
  - To convert logits into probabilities

$$P(y_t|y_{<t}, x) = \frac{e^{\text{logits}_t}}{\sum_{v=1}^{V} e^{\text{logits}_{t,v}}}$$

    - Applied along the vocabulary dimension (-1)
    - Output shape remains: $P \in \mathbb{R}^{(B,T_y,V)}$

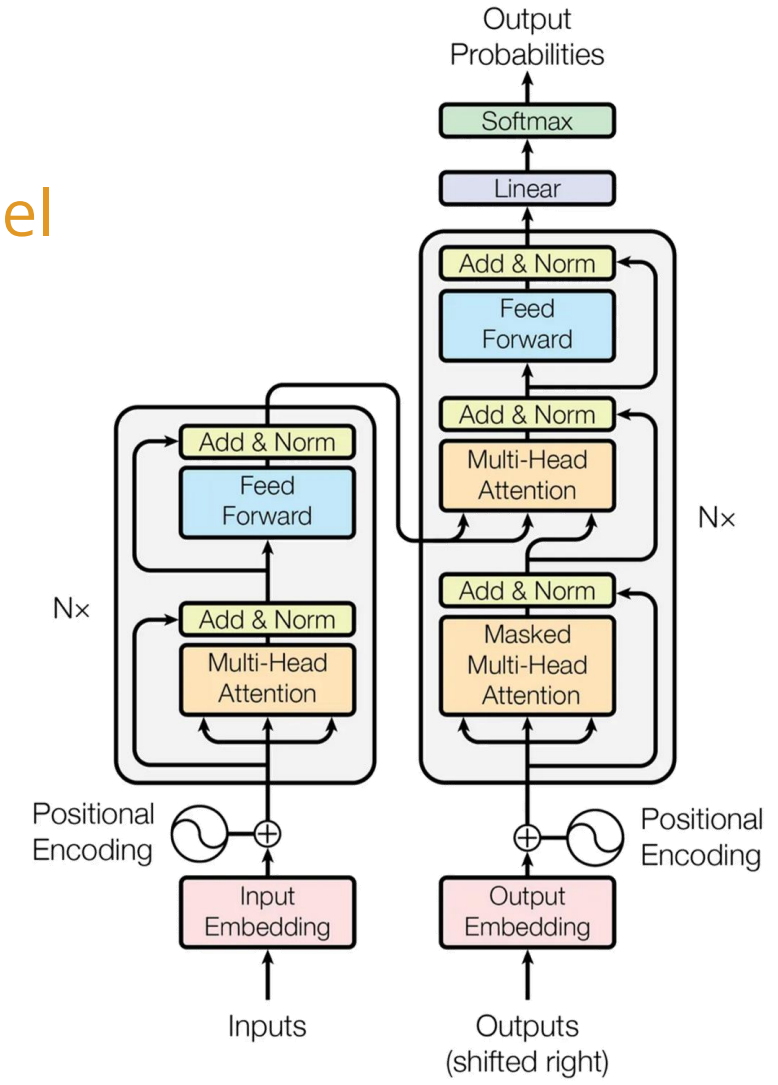


# Transformer architecture: Decoder – Last layers

- **Training vs inference**
  - Training
    - We compute probabilities for all tokens in parallel
    - **Loss**: average cross-entropy between predicted probabilities and ground truth tokens
  - Inference
    - We only need probabilities for the next token
    - We select the token with

$$y_t = \arg\max_v P(v|y_{<t}, x)$$

This is greedy decoding, sometimes suboptimal



# Transformer architecture: Decoder – Last layers

**Note on inference strategies**

- Greedy decoding (often suboptimal)
  - Local maximum at each step may lead to poor overall sequence
- Beam search
  - Keeps the *top-k* partial hypotheses at each time step (beam width = *k*)
  - Expands each by one possible next token
  - Scores all candidate by their cumulative log-probability

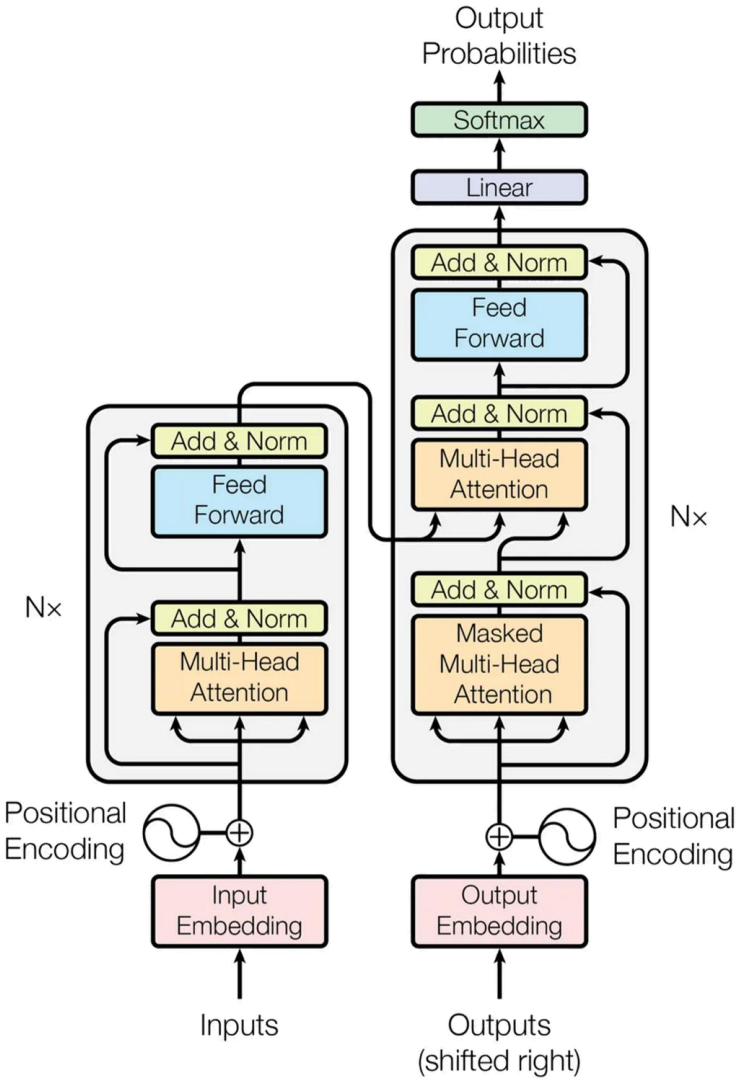
$$\text{score}(y_{1:t}) = \sum_{i=1}^{t} \log P(y_i | y_{<i}, x)$$

  - Keeps the *k* best sequences for the next step
  - After the final token, pick the best-scoring full sentence

# Transformer architecture: Decoder – Last layers

- **Flow through the decoder (summary)**

| Operation | Notes |
|---|---|
| **Input Embeddings + Positional Encoding** | Each target token represented as a vector |
| **Masked Multi-Head Self-Attention** | Each token attends to past tokens only |
| **Add & Norm** | Residual connection preserves dimension |
| **Cross-Attention (Encoder–Decoder)** | Decoder tokens attend to encoder outputs |
| **Add & Norm** | Stabilizes representation |
| **Feedforward Network (FFN)** | Applied independently per token |
| **Add & Norm** | Output passed to next decoder layer |
| **Final Linear Projection** | Maps each token vector to vocab logits |
| **Softmax** | Converts logits to probabilities |



# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Training and inference tips

- Transformers are powerful but sensitive to optimization choices
  - Training requires careful data handling, initialization and regularization
    - Tokenization
    - Padding
    - Batching
    - Warm-up
    - Regularization
    - Handling long sequences
    - Gradient clipping
  - Inference requires efficient decoding and memory management
    - Decoding strategies
    - Padding

# Training and inference tips

- **Training tips**
  - Tokenization:
    - Use subword tokenizers for open vocabulary (e.g., BPE)
      - Word-level tokenization suffers from vocabulary explosion, out-of-vocabulary problems and memory inefficiency
      - Character-level tokenization suffers from slower training, and it is harder to learn meaningful patterns
      - Subword tokenizers (e.g., BPE, WordPiece…) split text into units between characters and words
        - “unbelievable” – “un”, “believe”, “able”
        - Frequent words stay intact (“dog”, “tree”)
        - Rare or unknown words are decomposed into familiar parts

# Training and inference tips

- **Training tips**
  - Padding
    - Transformers process sequences in parallel, so all sentences in a batch must have the same length
      - All tensors in  a batch are expected to have the same dimensions for efficient GPU computation
    - Since natural sentences vary, we use **padding tokens** (<PAD>)
      - Sentence 1: “Bilbo found the ring”
      - Sentence 2: “Gandalf arrived”
      - Lengths 4 vs 2, we can’t form a single $(B, T_x, d_{model})$ tensor
    - **Solution**: Padding all sequences in the batch to the maximum length
      - We use a padding mask ($-\infty$) before softmax to prevent the model from attending to <PAD> positions

# Training and inference tips

- **Training tips**
  - Batching
    - Group sentences by length to minimize padding and maximize GPU efficiency
  - Warm-up
    - Learning rate scheduling is crucial (warm-up strategy)
  - Regularization
    - Common techniques are dropout inside attention weights, FFN intermediate activations, and embeddings
    - Early stopping based on validation loss

# Training and inference tips

- **Training tips**
  - Handling long sequences
    - Self-attention costs $O(T^2)$ in memory/time
    - It is common to truncate or window sequences, or using other variants of attention (e.g., sparse or efficient attention)
  - Gradient clipping
    - Helps prevent exploding gradient in early training
    - Stabilizes training

# Training and inference tips

- **Inference tips**
  - Decoding strategies
    - Deterministic (for translation, summarization, captioning…)
      - Greedy decoding: fast but may miss better sentences
      - Beam search: balances quality and speed
    - Controlled randomness (for stories, dialogue, creative tasks…)
      - Sampling: Instead of $y_t = \arg\max_v P(v)$
        - We sample $y_t \sim P(v)$
          - Common tokens are still likely but rare tokens can occasionally appear
      - Temperature scaling: modify the logits to control randomness

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- T<1: Sharper softmax. More deterministic.
- T=1: Normal softmax. Default model probabilities
- T>1: Flatter softmax. More diverse

# Training and inference tips

- **Inference tips**
  - Padding
    - We generate one sequence at a time, so no padding is needed
    - But, when doing batched decoding, we still need to mask <PAD> tokens from shorter hypotheses
    - Padding mainly matters for parallel training but also for efficient batched inference if we decode multiple sentences together

# Outline

- Introduction
- Attention in RNNs
- Attention in Transformers
- Transformer architecture: Encoder
- Transformer architecture: Decoder
  - General description
  - Masked multi-head attention
  - Cross-attention
  - Last Layers
- Training and inference tips
- Other architectures

# Other architectures

- **Transformer variations**
  - The 2017 “Attention Is All You Need” paper introduced a sequence to sequence (encoder-decoder) Transformer
  - Since then, architectures have evolved:
    - Encoder-only: better for *understanding* (e.g., BERT)
      - Sentence/token classification, text similarity, masked LM…
    - Decoder-only: better for generation (e.g., GPT)
      - No cross-attention
      - Text generation, dialogue
    - Encoder-decoder: general purpose seq-to-seq (e.g., T5, BART)
      - Translation and general seq-to-seq